Technion, Institute of Technology, Haifa, Israel

PhD Thesis Proposal

Direct Load Balancing for Mixture of Experts using Straight-Through Estimator

by

Mikey Shechter

Advisor: **Prof. Daniel Soudry**

May, 2026

Abstract

Sparsely activated Mixture-of-Experts (MoE) layers increase model capacity by replacing dense feed-forward blocks with many experts while activating only a small subset for each token. This efficiency depends on balanced routing if a small number of experts receive most tokens, the remaining capacity is wasted and the overloaded experts become a computation and communication bottleneck. Modern MoE models therefore rely on explicit load-balancing mechanisms.

The standard auxiliary load balancing loss optimizes this goal only indirectly. The quantities that determine balance are the hard per-expert token counts, but these counts depend on the discrete top-K routing decision and are not differentiable. As a result, the standard loss routes gradients through soft routing probabilities, creating a differentiable proxy that can improve balance but also pushes routing distributions toward uniformity and can weaken expert specialization.

This proposal studies Direct Load Balancing (DLB), a load-balancing objective that optimizes the hard assignment counts directly. DLB keeps the exact top-K routing decisions in the forward pass and uses a rectangular-window straight-through estimator in the backward pass to supply gradients for the hard count indicators. This makes the true squared load-imbalance objective, as well as count-based metrics such as MaxVio, directly optimizable without passing the balancing signal through a soft-probability proxy. In language-model experiments, DLB matches or improves on the standard auxiliary loss in both validation loss and balance at small scale, under both local and global load balancing. The results also show the importance of choosing the right objective. When the STE supplies a biased surrogate rather than exact gradients, objectives that are equivalent under exact differentiation can induce very different optimization dynamics and lead to very different balance.

The remaining work is to scale DLB enough to obtain an informative downstream signal, and to explore extensions such as bandwidth schedules, noisy variants, and related indicator-based objectives.

Contents

1 Introduction

$$\hat{r}_e^{(t)} = \begin{cases} r_e^{(t)}, & r_e^{(t)} + b_e \in \mathrm{TopK}\left(\{r^{(t)} + b\}\right) \\ 0, & \text{otherwise} \end{cases}$$

$$g_e = \frac{\mathrm{TopK}\left(\{r^{(t)} + b\}\right)_e p_e}{\sum_{j=1}^{E} \mathrm{TopK}\left(\{r^{(t)} + b\}\right)_j p_j}$$

Sparsely activated Mixture-of-Experts (MoE) layers have become one of the main tools for scaling neural networks. By replacing a dense feed-forward layer with many smaller expert feed-forward networks and a router that activates only a small subset of them per token, an MoE layer increases a model's parameter count without a proportional increase in the per-token computation [45, 22, 9]. This design now underlies many of the strongest open large language models [64, 5, 17, 31, 6].

However, the efficiency and quality gains of MoE only materialize when the router actually spreads tokens across the experts. If only a few experts receive most of the token load, the remaining experts are wasted, and under expert parallelism the overloaded experts become a step-time bottleneck. In practice, training an MoE layer without an explicit load-balancing (LB) signal reliably leads to expert collapse where a few experts receive almost all tokens while the rest are rarely used [45]. A standard solution is the auxiliary LB loss introduced by Fedus et al. [9].

The auxiliary loss is optimized together with the language-modeling loss and encourages the router to distribute token load evenly across experts. It is defined using two per-expert quantities: P_e, the average routing probability assigned to expert e, and f_e, the fraction of tokens actually assigned to that expert. The loss couples these quantities by penalizing large values of P_e for experts with large f_e. Since f_e is determined by discrete token assignments, it is not differentiable with respect to the router parameters. In practice, gradients therefore flow through P_e, while f_e acts as a weight that indicates which experts are overloaded. This mechanism reduces probability mass assigned to overloaded experts and pushes future routing decisions toward a more balanced load. This auxiliary loss is still widely used in MoE architectures today [64, 31, 55].

The motivation for this work is that the standard auxiliary loss is only a proxy for the true balance objective. Although it produces balanced routing, Guo et al. [11] observed that it can also drive the router toward more uniform per-token routing distributions, weakening expert specialization.

An alternative line of work avoids the auxiliary loss entirely by adjusting per-expert routing biases with a hand-designed update rule [52, 6]. This avoids the soft-probability proxy, but it replaces a learned gradient signal with a manual controller and does not directly address the specialization issue.

We instead aim to optimize the quantity that directly measures balance. To do so, we must pass gradients through f_e, which is a piecewise-constant function of the router logits. It has zero gradient almost everywhere and is discontinuous at routing boundaries. We use a straight-through estimator (STE) [1] for this purpose. The hard assignments are kept unchanged in the forward pass, while in the backward pass the derivative of the selection indicator is replaced by a localized rectangular-window surrogate, following the binarized-network STE of Hubara et al. [15]. With a gradient for the hard counts in hand, the true balance objective becomes directly optimizable. We call the resulting loss *Direct Load Balancing* (DLB).

Our small-scale language-model experiments show that DLB matches or improves on the standard auxiliary loss in both validation loss and balance. These gains hold for both local balancing and the global balancing setting of Qiu et al. [37].

We further show that the DLB gradient touches only tokens near routing boundaries, which helps preserve expert specialization (Section 3.4). Finally, because DLB only requires a gradient for

the hard counts, the same construction applies to a range of balance objectives. One notable example is the MaxVio balance metric introduced by Wang et al. [52], which reports only the most overloaded expert in each layer instead of the average balance across all experts. We define MaxVio and other direct LB objectives in Section 3.3 and evaluate them in Section 4.

Contributions.

- We introduce Direct Load Balancing, an MoE load-balancing loss that uses a rectangular-window STE to differentiate directly through the hard token-assignment counts, rather than through a soft-probability proxy.
- We analyze why DLB preserves expert specialization by only adjusting tokens near the routing boundary.
- We show that the same construction makes practical balance metrics, in particular MaxVio, directly optimizable, and we evaluate several DLB variants.
- We demonstrate that DLB matches or improves on the standard auxiliary loss in validation loss and balance at small scale, both for local and global balancing.

2 Related Work

Mixture-of-Experts. The sparsely gated Mixture-of-Experts layer of Shazeer et al. [45] was motivated by conditional computation: increasing the number of model parameters, and therefore the capacity to absorb large training corpora, without increasing the computation applied to every token by the same factor. This idea was adapted to transformer language models by GShard [22], and Switch Transformers [9] then scaled it further. Systems work such as DeepSpeed-MoE, Tutel, and Megatron Core made expert-parallel training and inference more practical [39, 16, 54], and recent open language models such as Mixtral, OLMoE, DeepSeekMoE, and Qwen3 show that MoE layers have become a standard large-model design choice [17, 31, 5, 55]. Most recent autoregressive MoE language models use a learned *token-choice* router: each token scores all experts and is sent to its top-K experts, possibly with additional engineering such as capacity limits [22, 9, 64], shared experts [5, 6], or grouped and node-limited routing [6, 54]. This routing rule requires an explicit way to balance tokens across experts, since otherwise training can collapse to a small subset of experts [45].

The auxiliary load-balancing loss. Shazeer et al. [45] already observed expert collapse and added explicit balance penalties, using separate losses on the coefficient of variation of the per-expert load and the per-expert importance. GShard used a related auxiliary loss together with capacity limits and token dropping [22], and Switch Transformers simplified the objective into the single auxiliary loss that is still the standard reference point [9]. ST-MoE kept the same basic balancing mechanism while adding stability improvements such as the router z-loss and recommending the now-common auxiliary coefficient [64]. Many later MoE language models still use this auxiliary loss or close variants, including OLMoE and Qwen3 [31, 55]. However, the standard auxiliary loss does not optimize the true balance objective directly, but only a differentiable proxy. This proxy comes with observable side effects, most notably pressure toward more uniform routing distributions and reduced expert specialization. DLB instead optimizes the count-based balance objective directly by differentiating through the hard assignments with an STE.

Loss-free balancing. Wang et al. [52] introduce the auxiliary-loss-free balancing strategy, later used in DeepSeek-V3, DeepSeek-V4, and in GLM-4.5 [6, 7, 58]. Instead of adding a differentiable

loss, their method adds a per-expert bias to the routing logits, increasing the bias for underloaded experts and decreasing it for overloaded experts. The appeal of this approach is that it uses a simple rule driven by the observed hard loads, rather than optimizing a soft-probability proxy. However, the bias is an expert-level correction that shifts the scores of all future tokens for that expert. It improves load balance, but it does not resolve the specialization issue because the same correction is applied to all tokens. DLB instead keeps the count-based balance objective as a differentiable training loss. With the rectangular-window STE, its gradient updates router parameters only through tokens near the top-K decision boundary, leaving confidently routed tokens untouched.

Balance-by-construction. Another way to avoid load imbalance is to make balance part of the routing rule itself. Expert-choice routing reverses the usual token-choice direction: each expert selects a fixed number of tokens from a block [63]. This guarantees balanced expert capacity, but raises two major issues. First, it breaks fixed per-token compute. Because the top-k selection is performed independently by each expert over the token block, a token may be selected by zero, one, or many experts. Second, the balance guarantee itself depends on computing those per-expert top-k sets over a block of tokens. In autoregressive generation, where the router must act causally on the current token rather than on a block containing future tokens, this routing rule is not directly available. BASE layers take a related approach: during training, they solve a balanced linear assignment problem so that each expert receives the same number of tokens [24]. They therefore suffer from the same block-level dependency. At test time, BASE falls back to greedy top-1 routing rather than using the balanced assignment rule. Hash layers avoid assignment solvers altogether by removing the learned router and using a fixed token-to-expert map [43]. This makes routing simple and often well balanced, but gives up learned, context-dependent expert selection. Overall, while exact balance solutions are attractive, they are not common in current autoregressive LLM MoE systems because of their practical issues.

Expert specialization. Expert specialization is an important and actively studied property of MoE language models. Different experts should acquire non-overlapping and focused knowledge rather than duplicate the same function [5, 26]. DeepSeekMoE explicitly designs for finer-grained and shared expert specialization [5], and OLMoE and recent MoE scaling studies emphasize that the useful number of experts and active experts depends on the training budget and target tasks [31, 21, 32]. Guo et al. [11] argue that the standard auxiliary loss can work against specialization by pushing routing distributions toward uniformity and increasing expert overlap, and add orthogonality and score-variance terms to counter this effect. Liu et al. [26] address homogeneous expert representations by modifying the optimizer to keep expert updates diverse, while Wei et al. [53] normalize gating logits to improve expert diversification. These methods add extra objectives, optimizer steps, or router transformations on top of the problematic auxiliary-loss setup. In some cases this also adds explicit training-time compute or memory overhead. DLB instead changes the balance signal itself without adding computational overhead, removing one source of anti-specialization pressure. Its gradient is nonzero only for tokens whose routing score lies within a bandwidth of the top-K threshold, so confidently routed tokens are left untouched.

Straight-through estimators. Bengio et al. [1] proposed propagating gradients through discrete operations by substituting a surrogate in the backward pass, and Hubara et al. [15] introduced the rectangular-window STE for training binarized networks, the surrogate we adopt here. Qin et al. [36] later emphasized that the STE surrogate itself can be annealed during training, moving from an early estimator that provides nonzero gradients over a wider range of inputs, but has a larger mismatch with the hard forward function, to a later estimator with lower mismatch. An analogous schedule may be useful for our rectangular-window bandwidth, as discussed in Section 5.2. STE through the hard routing decision has been used to improve other aspects of MoE

training. SparseMixer and SparseMixer-v2 approximate the routing-gradient terms that standard sparse MoE training drops, giving the router some signal about the hard expert-choice decision without evaluating every expert [27, 28]. Panda et al. [34] keeps sparse forward computation, but gives the router dense backward feedback by replacing the missing outputs of non-selected experts with running-average defaults. These works apply surrogate gradients to the routing computation itself, while leaving the load-balancing objective in its standard soft-proxy form. We instead leave the routing computation unchanged and apply the STE to the load-balancing loss.

3 Method

In this section we describe our Direct Load Balancing (DLB) method. We introduce notation by going over the MoE layer and the standard auxiliary loss (Section 3.1), then define the DLB loss and its STE gradient (Section 3.2), discuss other direct balance variants (Section 3.3), and finally analyze why DLB promotes expert specialization (Section 3.4).

3.1 Preliminaries and notation

MoE layer. The Mixture of Experts (MoE) layer [45] is a sparsely activated layer that replaces the dense feed-forward (FFN) layer of the standard transformer [51] with E expert FFNs $\mathrm{FFN}_1, \ldots, \mathrm{FFN}_E$ and a router that selects which K of the E experts are activated for each input token. Given an input token representation $x \in \mathbb{R}^d$, the router $R : \mathbb{R}^d \to \mathbb{R}^E$ produces routing logits

$$r = (r_1, \ldots, r_E) = R(x) \in \mathbb{R}^E,$$

and routing probabilities

$$p = (p_1, \ldots, p_E) = \mathrm{softmax}(r) \in \mathbb{R}^E.$$

Let $\mathrm{TopK}(p) \in \{0,1\}^E$ be the selection vector for the K largest values of p, with ties broken arbitrarily. The normalized gate on expert e is

$$g_e = \frac{\mathrm{TopK}(p)_e \, p_e}{\sum_{j=1}^{E} \mathrm{TopK}(p)_j \, p_j},$$

and the MoE output for token x is

$$y = \sum_{e=1}^{E} g_e \, \mathrm{FFN}_e(x).$$

This gating mechanism ensures that each token only requires K active experts to produce the output y.

Auxiliary load-balance loss. The efficiency and performance gains of large-scale MoE models appear only when different tokens are routed to different experts, so the router must spread the batch reasonably evenly across experts. In practice, training without an explicit load-balancing (LB) signal often leads to expert collapse [45], where a few experts receive almost all tokens while the rest are rarely used. The standard solution is the auxiliary LB loss of Fedus et al. [9]. For a batch of T tokens $x^{(1)}, \ldots, x^{(T)}$, we denote the routing probabilities of token $x^{(t)}$ by $p^{(t)}$ and define

$$f_e = \frac{1}{T} \sum_{t=1}^{T} \mathrm{TopK}(p^{(t)})_e, \qquad P_e = \frac{1}{T} \sum_{t=1}^{T} p_e^{(t)}.$$

Here f_e is the fraction of tokens in the batch routed to expert e, while P_e is the average routing probability mass assigned to expert e across the batch. Since each token selects exactly K experts,

perfect balance means $f_e = K/E$ for every expert e. A natural LB objective is therefore to minimize

$$\mathcal{L}_{\text{bal}} = \sum_{e=1}^{E} \left(f_e - \frac{K}{E} \right)^2 . \tag{1}$$

Using the equality $\sum_{e=1}^{E} f_e = K$, we obtain

$$\min_{\theta} \sum_{e=1}^{E} \left(f_e - \frac{K}{E} \right)^2 = \min_{\theta} \left(\sum_{e=1}^{E} f_e^2 - 2\frac{K^2}{E} + \frac{K^2}{E} \right) = \min_{\theta} \sum_{e=1}^{E} f_e^2,$$

so the standard auxiliary loss instead targets the simplified objective

$$\sum_{e=1}^{E} f_e^2.$$

This objective is not directly differentiable, because f_e depends on the discrete top-K routing decision. The standard auxiliary loss handles this by replacing one factor of f_e with the routing probability mass P_e, which acts as a differentiable proxy for the expected load, and uses the surrogate

$$\sum_{e=1}^{E} f_e P_e.$$

It then multiplies this objective by the number of experts E, so its scale remains $O(1)$ as E changes, and by a coefficient α that balances the LB objective against the language-modeling loss. The auxiliary LB loss is then

$$\mathcal{L}_{\text{aux}} = \alpha \cdot E \cdot \sum_{e=1}^{E} f_e \, P_e, \tag{2}$$

with gradients flowing only through P_e. While this objective encourages balanced hard assignments f_e, it also pushes the average routing probabilities P_e toward uniformity. Guo et al. [11] argue that this additional pressure toward uniform routing causes the model to converge to more uniform per-token routing distributions and weakens expert specialization. In contrast, the DLB loss we introduce next depends only on the hard assignment statistics f_e, so it does not encourage uniform soft routing probabilities. We discuss this in Section 3.4.

3.2 Direct Load-Balance Loss

Our goal is to directly optimize the true balance objective in Eq. (1). Since

$$\nabla_\theta \mathcal{L}_{\text{bal}} = 2 \sum_{e=1}^{E} \nabla_\theta f_e \left(f_e - \frac{K}{E} \right),$$

it is enough to derive a good estimator for $\nabla_\theta f_e$. We start by rewriting the top-K routing rule as a threshold test:

$$\text{TopK}(p^{(t)})_e = \mathbb{1}_{\left\{ r_e^{(t)} - m(\mathbf{r}^{(t)}) \geq 0 \right\}},$$

where $\mathbf{r}^{(t)} = (r_1^{(t)}, \ldots, r_E^{(t)})$ are the routing logits of token $x^{(t)}$, $m(\mathbf{r}^{(t)})$ denotes their Kth largest coordinate, and $\mathbb{1}_{\{A\}}$ is the indicator function

$$\mathbb{1}_{\{A\}} = \begin{cases} 1 & \text{if } A, \\ 0 & \text{otherwise.} \end{cases}$$

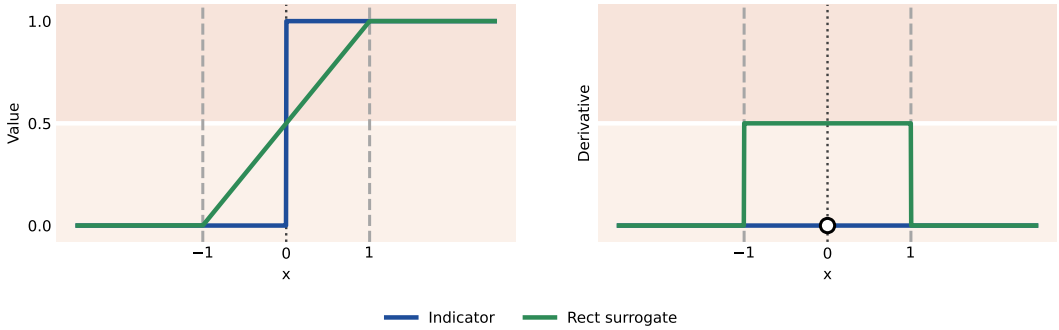

Figure 1: **Rectangular-window STE.** The left panel shows the hard indicator and the corresponding piecewise-linear surrogate for $h = 1$, and the right panel shows their derivatives. The forward pass uses the hard indicator for the top-K threshold test, while the backward pass replaces its derivative by a rectangular window around the threshold.

Using routing logits is equivalent to using routing probabilities here, since the softmax preserves the ordering of coordinates. We also note that if multiple experts tie at the threshold $m(\mathbf{r}^{(t)})$ we use the same arbitrary tie-breaking rule as the underlying top-K implementation, so exactly K experts are selected. Using this definition of TopK we get

$$f_e = \frac{1}{T}\sum_{t=1}^{T} \mathbb{1}_{\left\{r_e^{(t)} - m(\mathbf{r}^{(t)}) \geq 0\right\}}.$$

The obstacle is that the indicator has zero derivative almost everywhere and is undefined at the threshold. Following the straight-through estimator (STE) idea of Bengio et al. [1], and using the rectangular-window variant of Hubara et al. [15], we keep the exact hard routing decision in the forward pass but replace the backward derivative of the step function by a localized surrogate

$$\frac{d}{dz}\mathbb{1}_{\{z\geq 0\}} \approx \frac{1}{h}\,\mathrm{Rect}\left(\frac{z}{h}\right).$$

With

$$\mathrm{Rect}(u) = \mathbb{1}_{\{|u|<0.5\}},$$

where $h > 0$ is a bandwidth hyper-parameter. An illustration of the rect STE is shown in Figure 1.

Using the chain rule and applying this STE to f_e yields

$$\begin{aligned}\nabla_\theta f_e &= \frac{1}{T}\sum_{t=1}^{T} \frac{d}{dz}\mathbb{1}_{\{z\geq 0\}}\Big|_{z=r_e^{(t)}-m(\mathbf{r}^{(t)})} \nabla_\theta\left(r_e^{(t)} - m(\mathbf{r}^{(t)})\right) \\ &\approx \frac{1}{T}\sum_{t=1}^{T} \frac{1}{h}\,\mathrm{Rect}\left(\frac{r_e^{(t)} - m(\mathbf{r}^{(t)})}{h}\right)\nabla_\theta\left(r_e^{(t)} - m(\mathbf{r}^{(t)})\right).\end{aligned}$$

The remaining term $\nabla_\theta(r_e^{(t)} - m(\mathbf{r}^{(t)}))$ is differentiated normally by the chain rule, the STE only replaces the derivative of the indicator. Substituting this estimator into $\nabla_\theta \mathcal{L}_{\text{bal}}$ gives a direct gradient estimator for the true squared-load objective. We therefore define the *direct load-balance loss*

$$\mathcal{L}_{\text{DLB}} = \alpha \cdot E \cdot \sum_{e=1}^{E}\left(f_e - \frac{K}{E}\right)^2, \tag{3}$$

and use the following STE-based surrogate gradient in the backward pass:

$$\nabla_\theta \mathcal{L}_{\text{DLB}} \approx \frac{2\alpha \cdot E}{T}\sum_{e=1}^{E}\left(f_e - \frac{K}{E}\right)\sum_{t=1}^{T}\frac{1}{h}\,\mathrm{Rect}\left(\frac{r_e^{(t)} - m(\mathbf{r}^{(t)})}{h}\right)\nabla_\theta\left(r_e^{(t)} - m(\mathbf{r}^{(t)})\right), \tag{4}$$

with E being the same scaling factor and α the same coefficient as the regular auxiliary loss.

3.3 Direct load-balance variants

The construction above only requires a gradient estimator for the hard assignment statistics f_e, so it extends immediately to other balance objectives built from the same statistics.

MaxVio. A common way of evaluating the degree of balance of an MoE layer is the maximal violation (MaxVio) metric of Wang et al. [52]:

$$\mathrm{MaxVio} = \frac{\max_e f_e - K/E}{K/E}. \tag{5}$$

This metric isolates the most overloaded expert, which is often the step-time bottleneck under expert parallelism. Multiplying by α and dropping the additive constant gives the loss

$$\mathcal{L}_{\mathrm{MaxVio}} = \alpha \cdot \frac{E}{K} \max_e f_e,$$

where the E/K factor inherits from the metric's normalization and matches the dynamic range of $\mathcal{L}_{\mathrm{DLB}}$. Letting $e^* = \arg\max_e f_e$ and applying the rect STE yields

$$\nabla_\theta \mathcal{L}_{\mathrm{MaxVio}} \approx \frac{\alpha \cdot E}{K \cdot T} \sum_{t=1}^{T} \frac{1}{h} \mathrm{Rect}\left(\frac{r_{e^*}^{(t)} - m(\mathbf{r}^{(t)})}{h}\right) \nabla_\theta \left(r_{e^*}^{(t)} - m(\mathbf{r}^{(t)})\right).$$

In contrast to $\mathcal{L}_{\mathrm{DLB}}$, which applies pressure to every expert simultaneously, $\mathcal{L}_{\mathrm{MaxVio}}$ produces a gradient signal only on the single most-overloaded expert.

uncentered-DLB. A natural-looking variant is the uncentered-DLB balancing objective

$$\mathcal{L}_{\text{uncentered-DLB}} = \alpha \cdot E \cdot \sum_{e=1}^{E} f_e^2.$$

As noted in Section 3.1, $\mathcal{L}_{\text{uncentered-DLB}}$ and $\mathcal{L}_{\mathrm{DLB}}$ differ only by an additive constant, so under *exact* gradients they induce identical optimization dynamics. This equivalence relies on the identity $\sum_e \nabla_\theta f_e = 0$, which holds because $\sum_e f_e = K$ is constant. The STE surrogate, however, replaces $\nabla_\theta f_e$ with a biased estimate that does *not* satisfy this identity, so minimizing the two objectives yields different gradients. Differentiating $\mathcal{L}_{\text{uncentered-DLB}}$ and substituting the STE gives

$$\nabla_\theta \mathcal{L}_{\text{uncentered-DLB}} \approx \frac{2\alpha \cdot E}{T} \sum_{e=1}^{E} f_e \sum_{t=1}^{T} \frac{1}{h} \mathrm{Rect}\left(\frac{r_e^{(t)} - m(\mathbf{r}^{(t)})}{h}\right) \nabla_\theta \left(r_e^{(t)} - m(\mathbf{r}^{(t)})\right).$$

The two surrogate gradients differ only in their per-expert scaling: $\mathcal{L}_{\mathrm{DLB}}$ scales expert e's contribution by the signed deviation $(f_e - K/E)$, which vanishes at the target load and grows as f_e moves away from it in either direction, whereas $\mathcal{L}_{\text{uncentered-DLB}}$ scales by the non-negative load f_e itself. As a result, $\mathcal{L}_{\mathrm{DLB}}$ pushes overloaded experts down and underloaded experts up, penalizing both kinds of deviation symmetrically, while $\mathcal{L}_{\text{uncentered-DLB}}$ applies a one-directional pressure that pushes all loads downward, hardest on the most overloaded experts and weakest on those that are already underloaded.

The same distinction can be viewed as a correction to the raw STE estimate. Let

$$\nabla_\theta f_e = \frac{1}{T} \sum_{t=1}^{T} \frac{1}{h} \mathrm{Rect}\left(\frac{r_e^{(t)} - m(\mathbf{r}^{(t)})}{h}\right) \nabla_\theta \left(r_e^{(t)} - m(\mathbf{r}^{(t)})\right)$$

denote the raw STE estimate for $\nabla_\theta f_e$. Because every token is routed to exactly K experts, the load vector always satisfies $\sum_e f_e = K$. Therefore any exact collection of load gradients must satisfy

$$\sum_{e=1}^{E} \nabla_\theta f_e = \nabla_\theta \sum_{e=1}^{E} f_e = \nabla_\theta K = 0.$$

The raw STE estimates generally violate this constraint, since each expert's indicator is relaxed independently. A natural correction is therefore to remove the mean raw estimate and define

$$\widehat{\nabla_\theta f_e} = \nabla_\theta f_e - \frac{1}{E}\sum_{j=1}^{E} \nabla_\theta f_j.$$

This is the projection of the raw STE vector onto the constraint subspace $\sum_e g_e = 0$, so the corrected estimates again satisfy $\sum_e \widehat{\nabla_\theta f_e} = 0$.

$$\widehat{\nabla_\theta \mathcal{L}_{\text{uncentered-DLB}}} = \nabla_\theta \mathcal{L}_{\text{DLB}} = \widehat{\nabla_\theta \mathcal{L}_{\text{DLB}}}$$

Applying this correction to the uncentered objective exactly recovers the $\mathcal{L}_{\text{DLB}}$ surrogate gradient:

$$\begin{aligned}
\widehat{\nabla_\theta \mathcal{L}_{\text{uncentered-DLB}}} &= 2\alpha \cdot E \sum_{e=1}^{E} f_e \widehat{\nabla_\theta f_e} \\
&= 2\alpha \cdot E \sum_{e=1}^{E} f_e \left(\widetilde{\nabla_\theta f_e} - \frac{1}{E}\sum_{j=1}^{E} \widetilde{\nabla_\theta f_j} \right) \\
&= 2\alpha \cdot E \sum_{e=1}^{E} f_e \widetilde{\nabla_\theta f_e} - 2\alpha \sum_{e=1}^{E} f_e \sum_{j=1}^{E} \widetilde{\nabla_\theta f_j} \\
&= 2\alpha \cdot E \sum_{e=1}^{E} f_e \widetilde{\nabla_\theta f_e} - 2\alpha \cdot K \sum_{j=1}^{E} \widetilde{\nabla_\theta f_j} \\
&= 2\alpha \cdot E \sum_{e=1}^{E} \left(f_e - \frac{K}{E} \right) \widetilde{\nabla_\theta f_e}.
\end{aligned}$$

The last line is exactly the STE surrogate gradient obtained by differentiating $\mathcal{L}_{\text{DLB}}$ directly. Moreover, applying the same correction to $\mathcal{L}_{\text{DLB}}$ itself changes nothing, because $\sum_e (f_e - K/E) = 0$ cancels the subtracted mean term. Correcting the STE gradient and centering the objective are therefore equivalent, and we will see in Section 4 that $\mathcal{L}_{\text{DLB}}$ is the objective that balances well.

Other variants. For completeness we also evaluate two further variants. The first, $\mathcal{L}_{\text{TotalVio}}$, extends MaxVio from the single worst expert to the total absolute violation across experts:

$$\mathcal{L}_{\text{TotalVio}} = \frac{\alpha}{2} \cdot \frac{E}{K} \sum_{e=1}^{E} \left| f_e - \frac{K}{E} \right|.$$

The second, $\mathcal{L}_{\text{MaxVio}^2}$, is a squared version of $\mathcal{L}_{\text{MaxVio}}$:

$$\mathcal{L}_{\text{MaxVio}^2} = \frac{\alpha \cdot E^2}{(E-1)\,K^2} \left(\max_e f_e - \frac{K}{E} \right)^2.$$

The prefactors are chosen so that all of these objectives share the same dynamic range as the standard auxiliary loss. Together with $\mathcal{L}_{\text{MaxVio}}$, these span the natural combinations along the (sum vs. max) and (squared vs. absolute deviation) axes of the centered objective.

3.4 $\mathcal{L}_{\text{DLB}}$ promotes expert specialization

Because the STE introduces a bias, the dynamics that $\mathcal{L}_{\text{DLB}}$ induces during training are determined by its surrogate gradient rather than by the objective itself. We therefore reason about specialization by inspecting $\nabla_\theta \mathcal{L}_{\text{DLB}}$. Beyond the per-expert scaling $(f_e - K/E)$ already discussed in Section 3.3, the surrogate gradient differs from that of the standard auxiliary loss in *which tokens* contribute to balancing each expert.

Under the rect STE, the per-token contribution to $\nabla_\theta f_e$ is gated by $\text{Rect}\Big((r_e^{(t)} - m(\mathbf{r}^{(t)}))/h\Big)$, which is nonzero only for tokens whose score for expert e lies within a band of width h around the top-K threshold. Tokens that the router places clearly above or clearly below this threshold receive no gradient at all. This means that even for an overloaded expert, $\mathcal{L}_{\text{DLB}}$ does not penalize tokens that the router has decisively assigned to it or decisively rejected. It only adjusts the routing of tokens near the decision boundary. The symmetric statement holds for underloaded experts.

The standard auxiliary loss, by contrast, has its gradient flow through the average routing probability P_e, which is a smooth function of every token's routing logits. Balancing pressure is therefore applied to every token in the batch, pushing the per-token routing distributions toward uniformity and weakening expert specialization [11]. By restricting balancing pressure to boundary tokens, $\mathcal{L}_{\text{DLB}}$ leaves confidently routed tokens untouched and allows experts to specialize on their preferred inputs.

The bandwidth h acts as a knob on this mechanism: larger values spread the pressure across a wider band of tokens, while smaller values concentrate it on near-tied routing decisions and permit sharper specialization. Setting h too small, however, can leave too few tokens inside the rect window to produce a usable balancing signal, and can in turn degrade load balance. We examine this trade-off empirically in Section 4.

4 Experiments

We implement $\mathcal{L}_{\text{DLB}}$ and the other direct load-balance losses in the `modded-nanogpt-moe` repository[1], an MoE fork of `modded-nanogpt` [18]. We keep the repository defaults except for the MoE-specific settings: the backbone is a 12-layer transformer with hidden size $d = 768$, 6 attention heads, vocabulary size 50,304, sequence length 1024, rotary position embeddings [49], RMSNorm [60], QK-normalization [13], and ReLU^2 MLPs [47]. For optimization we follow the repository's split: AdamW [29] for the token embeddings and language-model head, with learning rates 0.3 and 0.002 respectively and betas $(0.9, 0.95)$, and Muon [19] for the remaining transformer parameters, with learning rate 0.02 and momentum 0.95. We use no warm-up, no weight decay, bfloat16 operations, and train on the C4 dataset [38].

For the small-scale experiments reported below we run on 8 A100 GPUs with micro-batch size 8 and 8 gradient-accumulation steps, keeping the global batch size at 512. We increase the number of experts to $E = 64$, to test load balancing in a setting closer to large-scale MoE runs, and correspondingly reduce the expert hidden dimension to 384. We experiment with both $K = 2$ and $K = 16$ active experts. These configurations have 559M total parameters and 120M and 219M active parameters, respectively. They are trained for 6000 iterations with 1714 linear warm-down iterations, for a total of 3.1B tokens.

Because low validation loss is unhelpful if routing is badly imbalanced, we select hyperparameters with a balance-aware rule. For each method we sweep the LB coefficient α, with the STE bandwidth fixed to $h = 1$ where applicable. The coefficient α sweep behind these selected configurations are reported in Appendix A.1, Tables A.1 to A.4 at fixed STE bandwidth $h = 1$ for both $K = 16$

[1] `https://github.com/cat-state/modded-nanogpt-moe`

Table 1: **Local load-balance results.** Validation loss and balance for the direct load-balance family and the standard auxiliary loss, at $E = 64$ experts with $K = 16$ (left) and $K = 2$ (right) active experts. The top row reports the configuration selected by the rule of Section 4. The bottom row reports the lowest-validation-loss run among configurations with MaxVio below 1.0. **Bold** marks the best and underline the second best in each column.

top-$K = 16$, MaxVio < 0.25

LB method	Validation Loss	MaxVio	TotalVio
$\mathcal{L}_{\text{DLB}}$	**3.031**	**0.043**	**0.654**
$\mathcal{L}_{\text{MaxVio}}$	3.166	0.067	2.989
$\mathcal{L}_{\text{uncentered-DLB}}$	3.064	0.463	10.969
$\mathcal{L}_{\text{TotalVio}}$	3.032	0.094	0.693
$\mathcal{L}_{\text{MaxVio}^2}$	3.033	0.119	6.353
$\mathcal{L}_{\text{aux}}$	**3.031**	0.086	2.022

top-$K = 2$, MaxVio < 0.25

LB method	Validation Loss	MaxVio	TotalVio
$\mathcal{L}_{\text{DLB}}$	**3.128**	0.169	0.978
$\mathcal{L}_{\text{MaxVio}}$	3.175	0.108	2.038
$\mathcal{L}_{\text{uncentered-DLB}}$	3.155	0.519	7.719
$\mathcal{L}_{\text{TotalVio}}$	3.403	0.093	1.775
$\mathcal{L}_{\text{MaxVio}^2}$	3.143	0.212	10.148
$\mathcal{L}_{\text{aux}}$	3.192	**0.028**	**0.538**

top-$K = 16$, MaxVio < 1.0

LB method	Validation Loss	MaxVio	TotalVio
$\mathcal{L}_{\text{DLB}}$	**3.028**	0.667	5.807
$\mathcal{L}_{\text{MaxVio}}$	3.029	0.627	30.126
$\mathcal{L}_{\text{uncentered-DLB}}$	3.064	0.463	10.969
$\mathcal{L}_{\text{TotalVio}}$	3.032	0.094	**0.693**
$\mathcal{L}_{\text{MaxVio}^2}$	3.029	0.376	19.856
$\mathcal{L}_{\text{aux}}$	3.031	**0.086**	2.022

top-$K = 2$, MaxVio < 1.0

LB method	Validation Loss	MaxVio	TotalVio
$\mathcal{L}_{\text{DLB}}$	**3.113**	0.565	9.584
$\mathcal{L}_{\text{MaxVio}}$	3.175	0.108	2.038
$\mathcal{L}_{\text{uncentered-DLB}}$	3.120	0.621	14.385
$\mathcal{L}_{\text{TotalVio}}$	3.122	0.337	1.865
$\mathcal{L}_{\text{MaxVio}^2}$	3.130	0.737	41.462
$\mathcal{L}_{\text{aux}}$	3.192	**0.028**	**0.538**

and $K = 2$. We also report preliminary bandwidth h sweeps in Tables A.5 to A.7, but those runs used a different environment, and we later found that they are not directly comparable to the runs reported here. Table 1 includes only runs from the shared environment, all with $h = 1$. These sweeps are partial because a full grid is computationally expensive. From each sweep we report the run with the lowest validation loss among configurations with MaxVio below 0.25 when such a configuration exists, and otherwise the run with the smallest MaxVio. The 0.25 threshold aligns with the capacity-factor convention used in Switch-style MoE systems: a capacity factor of 1.25 allocates capacity for a 25% overload relative to the uniform assignment [9, 64]. The analogy is not exact, since capacity factor controls allocated capacity per layer while MaxVio measures realized routing imbalance averaged across layers, but both express a similar overload margin. To show the effect of relaxing the balance requirement, we also report results selected with the looser constraint of MaxVio below 1.0.

4.1 Local load-balance results

Table 1 reports results when the assignment statistics f_e are computed locally, on each device's micro-batch. The top row uses the selection rule described in Sec. 4, showing experiments with MaxVio below 0.25, while the bottom row relaxes the requirement to MaxVio below 1.0 and then picks the lowest-validation-loss run for each method. Under the balance-aware rule, the straightforward $\mathcal{L}_{\text{DLB}}$ objective performs the best out of all the baselines and variants. For $K = 16$ it matches the standard auxiliary loss in validation loss while roughly halving MaxVio. When relaxing the the selection requirement to MaxVio below 1.0, it slightly improves performance, while we do not see the same improvement for the standard auxiliary loss. For $K = 2$ it improves over the baseline by 2%, and when relaxing the selection rule the improvement increases to 2.4% at the cost of allowing larger imbalance. Under the relaxed selection rule, $\mathcal{L}_{\text{MaxVio}}$ is also competitive on validation loss and on MaxVio, but because it only ever applies pressure to the single worst expert, it leaves the remaining experts comparatively imbalanced, which shows up as a 6x higher TotalVio compared to $\mathcal{L}_{\text{DLB}}$, while having very similar MaxVio. This supports our claim that we

Table 2: **Approximate global load-balance results.** Validation loss and balance under approximate global load balancing, at $E = 64$, $K = 16$, for two LB coefficients per method. The direct load-balance losses and the auxiliary loss use the approximate global computation of Qiu et al. [37], while the DeepSeek loss-free baseline uses an exact global token count. **Bold** marks the best and underline the second-best in each column.

LB method	LB coefficient α	Validation Loss	MaxVio	TotalVio
$\mathcal{L}_{\text{DLB}}$	0.01	<u>3.028</u>	0.049	<u>0.751</u>
	0.001	**3.027**	0.686	5.508
$\mathcal{L}_{\text{MaxVio}}$	0.01	3.056	**0.010**	**0.352**
	0.001	<u>3.028</u>	<u>0.038</u>	2.086
DeepSeek loss-free	0.01	3.029	0.068	1.215
	0.001	3.033	0.381	6.971
$\mathcal{L}_{\text{aux}}$	0.01	3.029	0.097	1.956
	0.001	3.033	0.934	16.824

can use different load balance losses to focus on different balance objectives.

Comparing the uncentered-DLB objective $\mathcal{L}_{\text{uncentered-DLB}}$ to the centered $\mathcal{L}_{\text{DLB}}$ isolates the effect of centering. The uncentered version has much worse balance, and a higher validation loss, even though the two objectives differ only by an additive constant under exact gradients. This gap appears because its STE gradient is built from a biased load-gradient estimate that does not satisfy the load-conservation identity $\sum_e \nabla_\theta f_e = 0$ discussed in Section 3.3. With the biased STE surrogate of Section 3.3, the exact-gradient equivalence between $\mathcal{L}_{\text{DLB}}$ and $\mathcal{L}_{\text{uncentered-DLB}}$ breaks drastically.

4.2 Approximate global load-balance results

The losses above are *local*. f_e is computed on each device's micro-batch, so balance is enforced over a small subset of tokens. A *global* LB loss Qiu et al. [37] instead computes f_e over the full global batch, pooling across data-parallel workers and gradient-accumulation steps. Global balancing is a strictly looser constraint because individual micro-batches may be imbalanced as long as the global batch is balanced. This leaves more room for expert specialization [37]. Because we use gradient accumulation in our small scale experiments, computing f_e exactly over the global batch requires an extra forward pass. Qiu et al. [37] suggested an alternative to the extra forward pass called *approximate* global LB. Approximate global LB maintains a running buffer that accumulates f_e across micro-batches. At each micro-batch we use the information in the buffer as f_e. At the end of the accumulation process, the information in the buffer matches the global f_e exactly. We also compare our results to the auxiliary-loss-free DeepSeek baseline of Wang et al. [52], which uses an exact global token count because their method does not require gradients, so it can accumulate exact counts, and only use them at the end of the global batch to update biases.

Table 2 reports results in this setting, sweeping α at $K = 16$. It shows that the direct load-balance losses are compatible with global balancing and benefit from it, just as the standard auxiliary loss does. As in the local setting, $\mathcal{L}_{\text{DLB}}$ has comparable to slightly better validation loss than the auxiliary loss and DeepSeek loss-free baselines, while also achieving better balance in both MaxVio and TotalVio. Compared with $\mathcal{L}_{\text{MaxVio}}$, $\mathcal{L}_{\text{DLB}}$ has comparable validation loss, and we again see the trade-off between worst-case balance and total balance, with $\mathcal{L}_{\text{MaxVio}}$ achieving lower MaxVio, while $\mathcal{L}_{\text{DLB}}$ keeping TotalVio lower.

Table 3: **Large-scale results.** Downstream CORE metrics, validation loss, and balance metrics for $E = 64$, $K = 8$ runs trained for 100B tokens. Mean CORE loss and CORE accuracy are averaged over the 20 evaluated DCLM CORE tasks. For each method, we report the run with the highest CORE accuracy among runs with MaxVio below 1.0. The full sweep is reported in Table A.8.

LB method	Mean CORE loss	CORE accuracy	Validation loss	MaxVio	TotalVio
$\mathcal{L}_{\text{aux}}$	5.451	0.274	2.849	0.857	9.312
DeepSeek loss-free	<u>5.263</u>	**0.290**	<u>2.755</u>	**0.041**	**0.678**
$\mathcal{L}_{\text{DLB}}$	**5.245**	<u>0.285</u>	**2.752**	<u>0.087</u>	<u>1.188</u>

4.3 Large-scale experiments

The small-scale experiments give an encouraging signal, but they do not by themselves show how the method behaves under longer training budgets. Scaling both model size and token count is expensive, so we keep the model scale close to the small-scale setting and scale primarily in tokens. This choice should make downstream evaluation more informative, rather than relying only on validation loss.

The large-scale configuration uses $E = 64$ experts and $K = 8$ active experts (559M total and 162M active parameters). We run this configuration on 32 H200 GPUs with micro-batch size 32 and no gradient accumulation, for a global batch size of 1024. Because there is no gradient accumulation, we use exact global load balancing rather than the approximate variant used in the small-scale global experiments. We train for 95,368 steps, for a total of 100B tokens. For the auxiliary-loss baseline and $\mathcal{L}_{\text{DLB}}$, we sweep $\alpha \in \{0.01, 0.001\}$. We also include two DeepSeek loss-free baselines: one with the standard softmax activation over router logits, and one with the sigmoid activation commonly used with loss-free routing. For $\mathcal{L}_{\text{DLB}}$, we sweep $h \in \{0.1, 0.5, 1.0, 2.0, 5.0\}$.

We evaluate our models on 20 of the 22 tasks in the CORE subset defined by DCLM [25]. This suite includes ARC-Easy and ARC-Challenge [4], six Big-Bench tasks [48] (QA Wikidata, Dyck languages, Operators, Repeat Copy Logic, CS Algorithms, and Language Identification), BoolQ [3], CommonsenseQA [50], COPA [42], CoQA [41], HellaSwag [57], LAMBADA [35], SQuAD [40], PIQA [2], Winograd Schema Challenge (WSC) [23], Winogrande [44], OpenBookQA [30], and AGI Eval LSAT-AR [62].

Table 3 shows that the large scale results are still not conclusive. As in the small scale experiments, we see that $\mathcal{L}_{\text{DLB}}$ has the lowest validation loss, with the loss-free DeepSeek baseline coming very close. Compared to the loss-free variant, it has a lower mean CORE loss, but also a lower CORE accuracy, and the results are very close each other and within the noise range. In terms of balance, the loss-free variant has a lower MaxVio as well as TotalVio, but both experiments are very well-balanced. The $\mathcal{L}_{\text{aux}}$ baseline performs poorly across the board with lower accuracy, higher loss, and worse balance.

Figure 2 gives a more detailed view of the large-scale training dynamics. The validation-loss gap between $\mathcal{L}_{\text{DLB}}$ and the loss-free baseline is small, but it is consistent throughout the run. For balance, the final MaxVio of the loss-free baseline is lower than that of $\mathcal{L}_{\text{DLB}}$, but $\mathcal{L}_{\text{DLB}}$ has lower MaxVio through most of training, with the ordering changing only near the end during learning-rate decay. We suspect this happens because the learning-rate decay does not apply to the loss-free bias updates, whose scale remains fixed while the optimizer updates become smaller. If so, the stronger balance of $\mathcal{L}_{\text{DLB}}$ during most of training could translate into faster training under expert parallelism, where imbalance affects the step-time bottleneck. This also suggests a possible hybrid strategy: turn off $\mathcal{L}_{\text{DLB}}$ during learning-rate decay and switch to loss-free balancing to apply a stronger final balancing correction. The two mechanisms appear independent enough that such a

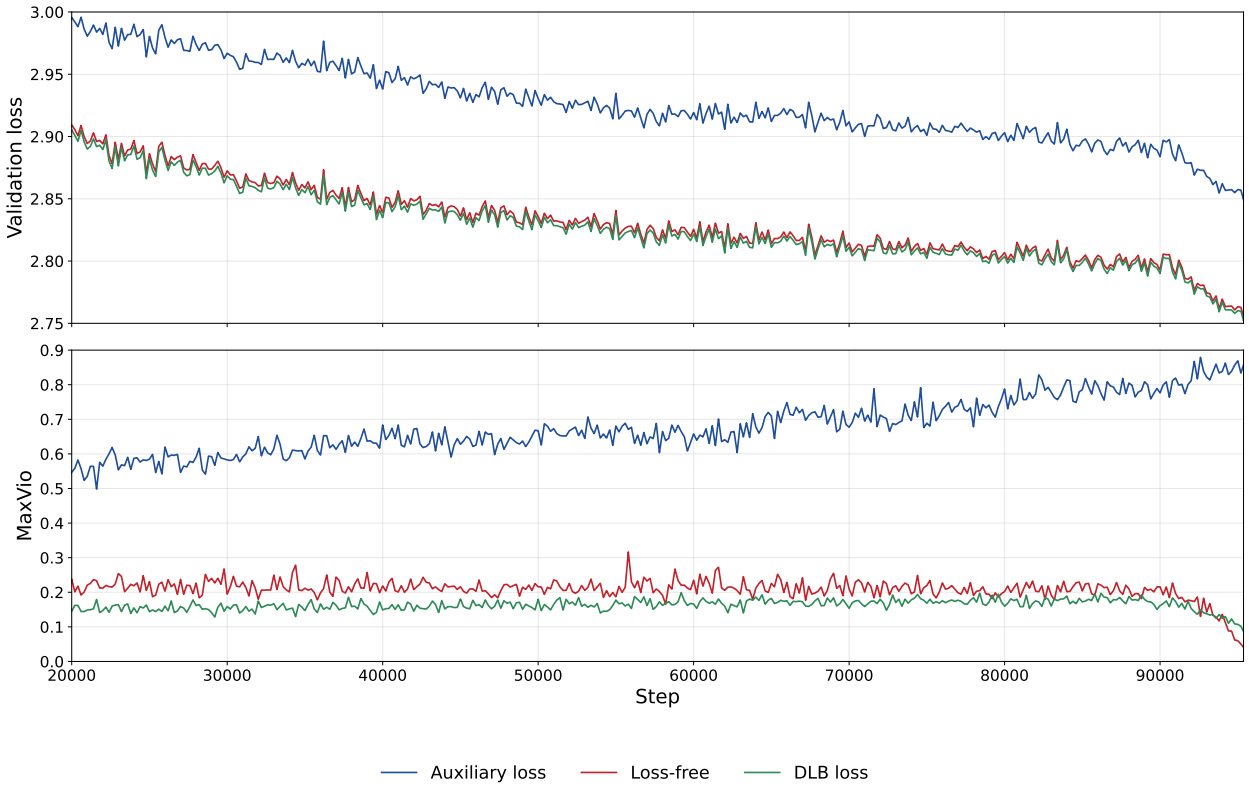

Figure 2: **Large-scale training dynamics.** Validation loss (top) and MaxVio (bottom) throughout the 100B-token runs.

switch may work, but this is only a hypothesis and should be tested empirically.

4.4 Bandwidth and the balance–specialization trade-off

Section 3.4 predicts that decreasing the STE bandwidth h can improve expert specialization, but only up to the point where enough tokens remain inside the rect window to provide a useful balancing signal. We study this trade-off for $\mathcal{L}_{\mathrm{DLB}}$ using the local load balance setting described in Section 4.1, by sweeping h and measuring router entropy together with TotalVio. Router entropy is a proxy for specialization: a confident router assigns most of its probability mass to a small number of experts and therefore has lower entropy, whereas a diffuse router has higher entropy. Entropy alone is not sufficient, because a collapsed or poorly balanced router will also have low entropy. We therefore report TotalVio alongside entropy to separate sharper specialization from routing imbalance.

Figure 3 shows that smaller bandwidths, from $h = 0.1$ to $h = 1.0$, have both lower router entropy and better balance than larger bandwidths such as $h = 5.0$ and $h = 10.0$. In this range, reducing h improves specialization without yet introducing a balance trade-off. At the smallest bandwidth, however, TotalVio begins to increase, which is consistent with the rect-window interpretation: when the window becomes too narrow, too few boundary tokens receive a balancing gradient.

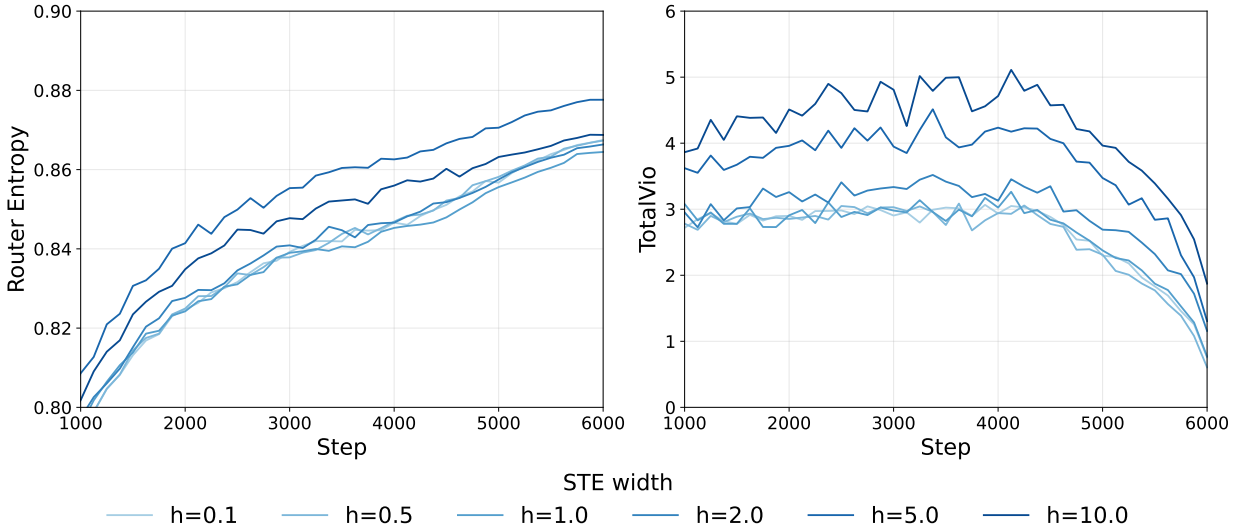

Figure 3: **Bandwidth and specialization.** Router entropy and TotalVio for $\mathcal{L}_{\text{DLB}}$ across STE bandwidth values. Smaller bandwidths reduce router entropy, indicating sharper routing decisions, but very small bandwidths can also weaken the load-balancing signal and increase imbalance.

5 Future work

5.1 Scaling up Direct Load Balancing

The most immediate continuation of this thesis is to scale up the Direct Load Balancing method. The experiments in Section 4 give encouraging but not yet conclusive evidence. At roughly 560M parameters and 3B training tokens, the centered $\mathcal{L}_{\text{DLB}}$ improves on the standard auxiliary loss and the DeepSeek loss-free baseline in validation loss while keeping high load balance. In the 100B-token runs at the same model scale, $\mathcal{L}_{\text{DLB}}$ achieves the lowest validation loss, but its downstream CORE accuracy is only comparable to the loss-free baseline. Validation loss alone is not a sufficient measure of large-scale model quality, so a conclusive result should show a clear improvement in downstream accuracy at larger scale.

We therefore plan to compare $\mathcal{L}_{\text{DLB}}$ against $\mathcal{L}_{\text{aux}}$ and loss-free load balancing at the scale of a few billion parameters. We will also vary the number of total experts and active experts to determine whether DLB is especially beneficial in particular routing regimes.

Scale also raises the question of how to set the bandwidth h of the rectangular-window straight-through estimator. With 64 total experts and 8 to 16 active experts, we find that h values between 0.1 and 1.0 perform similarly well, suggesting that the method is not very sensitive to this hyper-parameter in the current range. However, although we scale $\mathcal{L}_{\text{DLB}}$ by E so that the objective scale remains roughly consistent as the number of experts changes, this normalization may not be enough to make the best bandwidth scale-invariant. If h must change with model size or routing configuration, we would like to derive a small-scale predictive rule that avoids a large grid search at the target scale.

5.2 Scheduling the rect bandwidth

A related direction is to make the STE bandwidth h a training schedule rather than a fixed hyper-parameter. In the current method, h controls how far from the top-K threshold a token can be while still contributing to the load-balancing gradient. A larger h gives a denser and more stable balancing signal, but it also applies balancing pressure to tokens that are not especially close to

changing their expert assignment. A smaller h makes the rectangular surrogate more concentrated around the discontinuity of the step function, so the backward rule becomes closer to the hard routing decision used in the forward pass.

We therefore want to experiment with shrinking h over training. Early in training, a wide window would let the load-balancing objective update enough routing logits to prevent collapse and quickly find a balanced regime. Later in training, a narrower window would reduce the mismatch between the hard step used for routing and the surrogate used for the STE, and would leave confidently routed tokens increasingly untouched. This could preserve the early optimization benefits of a broad STE while ending with a sharper, more faithful estimator that better supports expert specialization.

This idea is inspired by the Error Decay Estimator of Qin et al. [36], which trains binary neural networks by annealing the backward surrogate for a hard sign function so that it becomes closer to the step-like forward function over time. For DLB, one option is to keep our rectangular-window STE and schedule only its bandwidth h. Another is to test their tanh-style surrogate directly, replacing the rectangular derivative by a temperature-controlled smooth approximation whose slope increases during training.

5.3 Noisy Direct Load Balancing

A work in progress direction is to turn the rectangular-window STE of Section 3.2 into an explicit noisy load-balancing objective. For a token $x^{(t)}$, write the top-K centered logit of expert e as

$$z_e^{(t)} = r_e^{(t)} - m(\mathbf{r}^{(t)}).$$

Instead of viewing the STE window only as a gradient estimation backward rule, we can view it as the derivative of a hard routing indicator after adding small noise to the top-K centered logits.

Let $\varepsilon_e^{(t)}$ be independent noise, with a different noise draw for each token, and write $\varepsilon^{(t)}$ for the noise associated with token t. We define the noise-averaged indicator

$$\sigma(u) = \mathbb{E}_\varepsilon \mathbb{1}_{\{u+\varepsilon \geq 0\}}.$$

For example, following the noise range in the derivation above, if $\varepsilon \sim U[-h, h]$, then

$$\sigma(u) = \begin{cases} 0 & u < -h, \\ (u+h)/(2h) & |u| \leq h, \\ 1 & u > h, \end{cases} \qquad \sigma'(u) = \frac{1}{2h} \operatorname{Rect}\Big(\frac{u}{2h}\Big).$$

Thus the rectangular STE can be interpreted as the derivative of a uniformly noised top-K centered logits.

The noised load of expert e would be

$$f_{e,\varepsilon} = \frac{1}{T} \sum_{t=1}^{T} \mathbb{1}_{\left\{z_e^{(t)} + \varepsilon_e^{(t)} \geq 0\right\}}.$$

For compactness, denote the noisy threshold indicator by

$$I_{e,\varepsilon}^{(t)} = \mathbb{1}_{\left\{z_e^{(t)} + \varepsilon_e^{(t)} \geq 0\right\}}, \qquad \mathbb{E}_{\varepsilon^{(t)}} I_{e,\varepsilon}^{(t)} = \sigma(z_e^{(t)}).$$

If we take the expectation of the direct load balance loss over the noise, we obtain

$$\mathcal{L}_{\text{noisy-DLB}} = \alpha \cdot E \cdot \mathbb{E}_\varepsilon \sum_{e=1}^{E} \left(f_{e,\varepsilon} - \frac{K}{E} \right)^2.$$

Expanding this expectation in the same way as the noiseless centered loss gives

$$\begin{aligned}\frac{\mathcal{L}_{\text{noisy-DLB}}}{\alpha \cdot E} &= \mathbb{E}_{\varepsilon} \sum_{e=1}^{E} \left(f_{e,\varepsilon} - \frac{K}{E} \right)^2 \\ &= \sum_{e=1}^{E} \left[\mathbb{E}_{\varepsilon} f_{e,\varepsilon}^2 - 2\frac{K}{E} \mathbb{E}_{\varepsilon} f_{e,\varepsilon} + \frac{K^2}{E^2} \right] \\ &= \sum_{e=1}^{E} \left[\frac{1}{T^2} \sum_{t=1}^{T} \sum_{t'=1}^{T} \mathbb{E}_{\varepsilon^{(t)},\varepsilon^{(t')}} \left[I_{e,\varepsilon}^{(t)} I_{e,\varepsilon}^{(t')} \right] - \frac{2K}{E \cdot T} \sum_{t=1}^{T} \mathbb{E}_{\varepsilon^{(t)}} I_{e,\varepsilon}^{(t)} + \frac{K^2}{E^2} \right].\end{aligned}$$

To separate the product expectation, we use that each $I_{e,\varepsilon}^{(t)}$ is an indicator and that token noises are independent:

$$\mathbb{E}_{\varepsilon^{(t)}} \left[\left(I_{e,\varepsilon}^{(t)} \right)^2 \right] = \mathbb{E}_{\varepsilon^{(t)}} I_{e,\varepsilon}^{(t)}, \qquad \mathbb{E}_{\varepsilon^{(t)},\varepsilon^{(t')}} \left[I_{e,\varepsilon}^{(t)} I_{e,\varepsilon}^{(t')} \right] = \mathbb{E}_{\varepsilon^{(t)}} I_{e,\varepsilon}^{(t)} \mathbb{E}_{\varepsilon^{(t')}} I_{e,\varepsilon}^{(t')} \quad (t \neq t').$$

Using these identities, the product expectation separates into diagonal and off-diagonal contributions:

$$\begin{aligned}\frac{\mathcal{L}_{\text{noisy-DLB}}}{\alpha \cdot E} &= \sum_{e=1}^{E} \Bigg[\frac{1}{T^2} \sum_{t=1}^{T} \mathbb{E}_{\varepsilon^{(t)}} I_{e,\varepsilon}^{(t)} + \frac{1}{T^2} \sum_{t=1}^{T} \sum_{\substack{t'=1 \\ t' \neq t}}^{T} \mathbb{E}_{\varepsilon^{(t)}} I_{e,\varepsilon}^{(t)} \mathbb{E}_{\varepsilon^{(t')}} I_{e,\varepsilon}^{(t')} \\ &\qquad - \frac{2K}{E \cdot T} \sum_{t=1}^{T} \mathbb{E}_{\varepsilon^{(t)}} I_{e,\varepsilon}^{(t)} + \frac{K^2}{E^2} \Bigg] \\ &= \sum_{e=1}^{E} \Bigg[\frac{1}{T^2} \sum_{t=1}^{T} \left(\mathbb{E}_{\varepsilon^{(t)}} I_{e,\varepsilon}^{(t)} - \left(\mathbb{E}_{\varepsilon^{(t)}} I_{e,\varepsilon}^{(t)} \right)^2 \right) + \frac{1}{T^2} \sum_{t=1}^{T} \sum_{t'=1}^{T} \mathbb{E}_{\varepsilon^{(t)}} I_{e,\varepsilon}^{(t)} \mathbb{E}_{\varepsilon^{(t')}} I_{e,\varepsilon}^{(t')} \\ &\qquad - \frac{2K}{E \cdot T} \sum_{t=1}^{T} \mathbb{E}_{\varepsilon^{(t)}} I_{e,\varepsilon}^{(t)} + \frac{K^2}{E^2} \Bigg] \\ &= \frac{1}{T^2} \sum_{e=1}^{E} \sum_{t=1}^{T} \sigma(z_e^{(t)}) \left(1 - \sigma(z_e^{(t)}) \right) + \sum_{e=1}^{E} \left(\frac{1}{T} \sum_{t=1}^{T} \sigma(z_e^{(t)}) - \frac{K}{E} \right)^2.\end{aligned}$$

The subtraction of $\left(\mathbb{E}_{\varepsilon^{(t)}} I_{e,\varepsilon}^{(t)} \right)^2$ in the first term comes from rewriting the off-diagonal sum as a full double sum minus its diagonal terms.

The final line shows that the noise-averaged loss contains two terms. The second term is the regular objective of $\mathcal{L}_{\text{DLB}}$, but applied to the expected noised loads

$$\frac{1}{T} \sum_{t=1}^{T} \sigma(z_e^{(t)}).$$

The first term is the variance of the noised expert loads, summed over experts:

$$\begin{aligned}\mathrm{Var}_\varepsilon(f_{e,\varepsilon}) &= \mathrm{Var}_\varepsilon\left(\frac{1}{T}\sum_{t=1}^{T} I_{e,\varepsilon}^{(t)}\right) \\ &= \frac{1}{T^2}\,\mathrm{Var}_\varepsilon\left(\sum_{t=1}^{T} I_{e,\varepsilon}^{(t)}\right) \\ &= \frac{1}{T^2}\sum_{t=1}^{T} \mathbb{E}_{\varepsilon^{(t)}} I_{e,\varepsilon}^{(t)}\left(1-\mathbb{E}_{\varepsilon^{(t)}} I_{e,\varepsilon}^{(t)}\right) \\ &= \frac{1}{T^2}\sum_{t=1}^{T} \sigma(z_e^{(t)})\left(1-\sigma(z_e^{(t)})\right).\end{aligned}$$

The variance separates across tokens because the noises are independent, and each noisy indicator is Bernoulli with mean $\mathbb{E}_{\varepsilon^{(t)}} I_{e,\varepsilon}^{(t)} = \sigma(z_e^{(t)})$. The variance term becomes smaller for tokens farther from the routing boundary: for uniform noise, once the centered logit is outside the noise range, $\sigma(z_e^{(t)})$ is exactly 0 or 1, so one of the two factors in $\sigma(z_e^{(t)})(1-\sigma(z_e^{(t)}))$ is zero. Intuitively, if small noise cannot change whether the indicator is active, then the noised load has zero variance from that token.

This gives an intuitive interpretation of the objective. Minimizing the second term is the usual $\mathcal{L}_{\text{DLB}}$ pressure toward balanced expected loads. Minimizing the first term additionally pushes tokens away from the top-K boundary, because the variance is largest for unstable near-boundary indicators. This extra pressure may improve expert specialization, but if it is too strong it can hurt load balance by preferring stable assignments over moving boundary tokens toward underloaded experts. This tradeoff can be controlled with separate coefficients, for example α_{bal} for the expected-load term and α_{var} for the variance term, and by sweeping their ratio.

Implementation options

The derivation above defines a noisy-DLB objective. There are several ways to connect that objective to training, depending on what we do in the MoE forward pass and what we use only for the load-balancing gradient.

1. **Sampled noisy MoE forward.** Use $I_{e,\varepsilon}^{(t)}$ to choose which experts to run. This is the straightforward noisy forward pass, but it no longer has a fixed top-K compute budget. Some tokens can activate more than K experts, while some can activate less. The issue with this variant is that the language-modeling loss backward pass also requires the forward value $\mathrm{FFN}_e(x^{(t)})$ for every active expert, so the extra experts have to be evaluated. This might add too much extra compute to be fairly compared to strict top-K methods.

2. **Deterministic MoE forward, analytic noisy-DLB auxiliary.** To avoid the extra expert computation of the sampled noisy forward pass, we can run the usual deterministic top-K MoE forward pass, and use $\mathcal{L}_{\text{noisy-DLB}}$ only as an auxiliary loss. No extra expert evaluations are needed, because this loss depends on router logits and load indicators, not on expert outputs. A possible issue with this variant is that there is a mismatch between what the auxiliary loss balances
$$\frac{1}{T}\sum_t \sigma(z_e^{(t)}),$$
and the actual MoE forward pass which uses the deterministic top-K assignments
$$f_e = \frac{1}{T}\sum_{t=1}^{T} \mathbb{1}_{\left\{z_e^{(t)}\geq 0\right\}}.$$

Here also, the objective adds the variance term

$$\frac{1}{T^2}\sum_{e=1}^{E}\sum_{t=1}^{T}\sigma(z_e^{(t)})\left(1-\sigma(z_e^{(t)})\right).$$

3. **Deterministic MoE forward, DLB STE plus variance.** A third option is to not add noise during training. Instead, use the analysis above only to add the variance term to the existing $\mathcal{L}_{\text{DLB}}$ objective. The MoE forward pass stays deterministic top-K, and the load term stays the usual hard DLB load

$$f_e = \frac{1}{T}\sum_{t=1}^{T}\mathbb{1}_{\left\{z_e^{(t)}\geq 0\right\}}.$$

Its backward pass uses the same rectangular STE as $\mathcal{L}_{\text{DLB}}$ in Section 3.2. The only added term is the deterministic variance regularizer

$$\frac{1}{T^2}\sum_{e=1}^{E}\sum_{t=1}^{T}\sigma(z_e^{(t)})\left(1-\sigma(z_e^{(t)})\right).$$

This option keeps the same forward assignments and the same DLB STE for the load term. The noisy analysis only supplies the extra variance penalty.

Noise placement

The derivation above adds noise after the top-K threshold is computed:

$$I_{e,\varepsilon}^{(t)} = \mathbb{1}_{\left\{r_e^{(t)}-m(\mathbf{r}^{(t)})+\varepsilon_e^{(t)}\geq 0\right\}}.$$

Here the top-K threshold is fixed before adding noise. For one noise draw, the number of active experts is

$$\sum_{e=1}^{E} I_{e,\varepsilon}^{(t)},$$

which can be smaller or larger than K.

Now consider a different placement: add noise to the router logits before computing the top-K threshold. Define

$$\widetilde{\mathbf{r}}^{(t)} = \mathbf{r}^{(t)} + \varepsilon^{(t)}, \qquad \widetilde{r}_e^{(t)} = r_e^{(t)} + \varepsilon_e^{(t)}.$$

Then the noisy indicator becomes

$$J_{e,\varepsilon}^{(t)} = \mathbb{1}_{\left\{\widetilde{r}_e^{(t)}-m(\widetilde{\mathbf{r}}^{(t)})\geq 0\right\}}.$$

This construction guarantees that

$$\sum_{e} J_{e,\varepsilon}^{(t)} = K.$$

The noised load is

$$f_{e,\varepsilon}^{\text{logit}} = \frac{1}{T}\sum_{t=1}^{T} J_{e,\varepsilon}^{(t)}.$$

For compactness, write the expected noisy top-K indicator as

$$\pi_e^{(t)} = \mathbb{E}_{\varepsilon^{(t)}} J_{e,\varepsilon}^{(t)} = \Pr_{\varepsilon^{(t)}}\left[\widetilde{r}_e^{(t)} \text{ is among the top-}K \text{ entries of } \widetilde{\mathbf{r}}^{(t)}\right].$$

If we repeat the same expected-loss analysis, the noised-logit DLB objective is

$$\mathcal{L}^{\text{logit}}_{\text{noisy-DLB}} = \alpha \cdot E \cdot \mathbb{E}_{\varepsilon} \sum_{e=1}^{E} \left(f^{\text{logit}}_{e,\varepsilon} - \frac{K}{E} \right)^2 .$$

Expanding gives

$$\frac{\mathcal{L}^{\text{logit}}_{\text{noisy-DLB}}}{\alpha \cdot E} = \sum_{e=1}^{E} \left[\mathbb{E}_{\varepsilon} \left(f^{\text{logit}}_{e,\varepsilon} \right)^2 - 2\frac{K}{E} \mathbb{E}_{\varepsilon} f^{\text{logit}}_{e,\varepsilon} + \frac{K^2}{E^2} \right].$$

The mean load is

$$\mathbb{E}_{\varepsilon} f^{\text{logit}}_{e,\varepsilon} = \frac{1}{T} \sum_{t=1}^{T} \pi_e^{(t)}.$$

The off-diagonal token terms factorize because the full noise vectors $\varepsilon^{(t)}$ are independent across tokens. This does not assume that $J^{(t)}_{e,\varepsilon}$ depends only on $\varepsilon^{(t)}_e$; for a fixed token, it depends on all expert noises in $\varepsilon^{(t)}$. Thus

$$\begin{aligned} \mathbb{E}_{\varepsilon} \left(f^{\text{logit}}_{e,\varepsilon} \right)^2 &= \frac{1}{T^2} \sum_{t=1}^{T} \mathbb{E}_{\varepsilon^{(t)}} J^{(t)}_{e,\varepsilon} + \frac{1}{T^2} \sum_{\substack{t,t'=1 \\ t \neq t'}}^{T} \mathbb{E}_{\varepsilon^{(t)}} J^{(t)}_{e,\varepsilon} \mathbb{E}_{\varepsilon^{(t')}} J^{(t')}_{e,\varepsilon} \\ &= \frac{1}{T^2} \sum_{t=1}^{T} \pi_e^{(t)} \left(1 - \pi_e^{(t)} \right) + \left(\frac{1}{T} \sum_{t=1}^{T} \pi_e^{(t)} \right)^2 . \end{aligned}$$

Therefore

$$\frac{\mathcal{L}^{\text{logit}}_{\text{noisy-DLB}}}{\alpha \cdot E} = \frac{1}{T^2} \sum_{e=1}^{E} \sum_{t=1}^{T} \pi_e^{(t)} \left(1 - \pi_e^{(t)} \right) + \sum_{e=1}^{E} \left(\frac{1}{T} \sum_{t=1}^{T} \pi_e^{(t)} - \frac{K}{E} \right)^2 .$$

So we get the same variance-plus-balance form as before, with $\sigma(z_e^{(t)})$ replaced by $\pi_e^{(t)}$. The difference is in the expected indicator. For centered-logit noise, the expected indicator is the scalar probability $\sigma(z_e^{(t)})$. For logit noise before top-K, the expected indicator is the noisy top-K selection probability

$$\pi_e^{(t)} = \Pr_{\varepsilon^{(t)}} \left[\widetilde{r}_e^{(t)} \text{ is among the top-}K \text{ entries of } \widetilde{\mathbf{r}}^{(t)} \right] = \Pr \left[\sum_{j \neq e} \mathbb{1}_{\left\{ \widetilde{r}_j^{(t)} > \widetilde{r}_e^{(t)} \right\}} \leq K - 1 \right].$$

This probability depends on all logits, because changing one expert's logit changes the probability that other experts enter the top-K set. Equivalently, conditioning on the noised value of expert e gives

$$\pi_e^{(t)} = \int p_{\varepsilon} \left(y - r_e^{(t)} \right) \Pr \left[\sum_{j \neq e} B_j^{(t)}(y) \leq K - 1 \right] dy,$$

where

$$B_j^{(t)}(y) \sim \text{Bernoulli} \left(\Pr_{\varepsilon_j^{(t)}} [r_j^{(t)} + \varepsilon_j^{(t)} > y] \right).$$

This expression is no longer the scalar function of $z_e^{(t)}$ that gave the rectangular STE. To use this analytic objective, we would need to compute or approximate these noisy top-K selection probabilities and differentiate through them. This is too expensive to be used practically in MoE training.

If instead we sample $J_{e,\varepsilon}^{(t)}$ once and use it only for the auxiliary loss, we avoid computing $\pi_e^{(t)}$. The sampled auxiliary loss would be

$$\widehat{\mathcal{L}_{\text{noisy-DLB}}}^{\text{logit}} = \alpha \cdot E \sum_{e=1}^{E} \left(f_{e,\varepsilon}^{\text{logit}} - \frac{K}{E} \right)^2 .$$

This brings us back to a discrete objective, because $J_{e,\varepsilon}^{(t)}$ is still a hard top-K indicator. With the true derivative, the gradient through $J_{e,\varepsilon}^{(t)}$ is zero almost everywhere. We could therefore apply the same rectangular STE as in $\mathcal{L}_{\text{DLB}}$, but now around the noisy top-K boundary:

$$\frac{\partial J_{e,\varepsilon}^{(t)}}{\partial \theta} \approx \frac{1}{h} \operatorname{Rect}\left(\frac{\widetilde{r}_e^{(t)} - m(\widetilde{\mathbf{r}}^{(t)})}{h} \right) \nabla_\theta \left(\widetilde{r}_e^{(t)} - m(\widetilde{\mathbf{r}}^{(t)}) \right).$$

But this means the noise only changes the sampled assignments and the location of the STE window. This option is therefore just a noisy version of $\mathcal{L}_{\text{DLB}}$, so we do not expect it to be useful in practice.

6 Discussion

The standard auxiliary loss $\mathcal{L}_{\text{aux}}$ was introduced as a differentiable proxy for load balance: it uses the average routing probability P_e because the hard assignment fraction f_e is not differentiable. This proxy is useful, but it is not the quantity we ultimately want to optimize. Direct Load Balancing instead uses a straight-through-estimator to keep the hard top-K assignment exact in the forward pass, and uses a surrogate derivative to optimize the hard per-expert token fractions directly. The rectangular-window STE supplies gradients for these discrete fractions, making the true count-based metrics such as the squared-imbalance objective and MaxVio, directly optimizable.

Empirically, $\mathcal{L}_{\text{DLB}}$ matches or improves on the standard auxiliary loss in both validation loss and balance, under local and global balancing alike. The behavior of the different direct objectives also matches the trade-off they are designed to express. Optimizing worst-case balance with $\mathcal{L}_{\text{MaxVio}}$ gives relatively weaker total balance, as reflected by TotalVio, while optimizing the total squared-imbalance objective $\mathcal{L}_{\text{DLB}}$ gives lower TotalVio but relatively higher MaxVio. This suggests that the direct load-balancing framework can target different operational notions of balance by changing the count-based objective.

The results also show the importance of choosing the right objective when using only an estimate of the true gradient. Under exact gradients, the centered and uncentered squared objectives are identical. Under the STE surrogate they are not, and empirically only the centered objective $\mathcal{L}_{\text{DLB}}$ balances well. Thus, algebraic equivalence under exact differentiation is not enough. With an STE, the surrogate gradient induced by the chosen objective matters.

More broadly, Direct Load Balancing is one instance of a general recipe: keep a non-differentiable, indicator-based training quantity exact in the forward pass and differentiate through it with an STE. Even within the MoE architecture, this recipe suggests several other targets. For example, we could optimize the load of the most overloaded expert across all layers rather than balancing each layer independently. We could incorporate token dropping directly into the training objective. We could also try to reduce cross-GPU communication when experts are distributed across devices during training or inference. These are exciting directions for future work.

The key limitation is scale. Our experiments are at the scale of a few hundred million parameters. Although the performance trends are consistent across the routing regimes we tested, confirming them at larger scale and obtaining a clear downstream-task signal remain essential.

7 Future Research Directions

7.1 Overview

Direct Load Balancing shows that a non-differentiable, indicator-based training quantity can be optimized directly by keeping the hard quantity exact in the forward pass and using a straight-through surrogate to supply the missing gradient. This section outlines how we intend to take that work forward.

We plan to apply the straight-through recipe developed in Section 3.2, where we keep a non-differentiable, indicator-based training quantity exact in the forward pass and differentiate through it with a surrogate gradient, to discrete operations beyond load balancing (Section 7.2). We single out the cross-device communication volume in expert-parallel Mixture-of-Experts, and the keep-or-drop decision in data filtering, as two indicator-based quantities that are currently optimized only indirectly.

7.2 Straight-through estimation for other discrete operations

The idea of Direct Load Balancing is to optimize some non-differentiable, indicator-based quantity that currently is only optimized indirectly. Load balance is a natural candidate for such an idea, especially considering that the standard auxiliary loss $\mathcal{L}_{\text{aux}}$, the current leading method, already optimizes a proxy for the indicator, but we believe we can find a better estimation than it. This idea can be used in several other applications.

Communication volume in expert-parallel MoE

Large MoE models are often trained and served with expert parallelism. The experts are sharded across devices, so when a token is routed to an expert on a different device, the token's representation must be sent across the interconnect [22, 39, 16]. This all-to-all communication can be very expensive when training large-scale language models. Systems such as FasterMoE, Tutel, and SmartMoE improve expert-parallel execution through communication and scheduling optimizations [12, 16, 59]. Other methods reduce the actual remote traffic by exploiting structure in the routed tokens and experts: LSH-MoE compresses similar token representations before all-to-all communication [33], ExFlow, MoETuner, and VELA choose communication-aware expert placements for inference, serving, or fine-tuning [56, 10, 14], and C2R constrains the activated experts of each token to a small collaboration group that can be located on the same device [61]. These papers show that expert-parallel communication is an active research problem, but they mostly address it through system-level communication-mitigation techniques.

Our proposed direction is to optimize the communication count directly during pretraining. At a high level, the objective is simple: if token t is currently on device d_t, and expert e is stored on device d_e, then routing t to e requires communication exactly when $d_t \neq d_e$. We can therefore add a communication objective

$$L_{\text{comm}} = \sum_{t} \sum_{e=1}^{E} \text{TopK}(p^{(t)})_e \mathbb{1}_{\{d_t \neq d_e\}}.$$

This is not differentiable because it depends on the hard top-K routing decision, but it has the same structure as the load-balance count in $\mathcal{L}_{\text{DLB}}$. The forward pass would use the exact communication count, and the backward pass would differentiate through the top-K threshold with the rectangular-window straight-through estimator of Section 3.2. The router would then learn a direct preference for low-communication routes, while the coefficient on L_{comm} would control the trade-off with the language-modeling loss and load balancing.

While this sounds straightforward, there are several technical difficulties. The first is token ownership. In the standard expert-parallel implementation, a token is dispatched to its selected experts and then the expert outputs are gathered back to the original owner device, so merely making consecutive experts live on the same device does not necessarily save communication. ExFlow addresses this by allowing the token context to follow the expert computation across layers, so inter-layer expert affinity can reduce all-to-all communication [56]. For training, we would need an analogous setup in which token ownership can migrate.

The second difficulty is that each token activates K experts but has only one owner. If all selected experts are on the same device, the objective can clearly prefer that device-local group. If the selected experts are split across several devices, no single owner eliminates all communication. In that case the objective should still reduce the total communication volume, hopefully by encouraging the router to choose experts that are co-located when possible. When co-location is not possible, the implementation could choose the next owner from among the selected expert devices, for example the device containing the largest number of the token's selected experts.

The third caveat is that the usual router sees the token representation but not the device currently holding that token, so a router without device information cannot consistently know which experts are local. A more explicit variant is to inject device information only into the router, for example by adding a learned source-device embedding or a device-specific router bias before computing the logits. This information should be recomputed from the token's current owner at each MoE layer and kept out of the expert input itself. If a token is moved to a different owner, the router receives the new device embedding rather than carrying the old one forward.

Direct data filtering with straight-through estimation

A second target is data filtering. In my earlier work, FLYT [46], a scoring model learns to weight each pretraining example by how useful it is for downstream performance, using gradient signals from downstream task datasets. FLYT produces continuous per-example weights, which are then used to filter the dataset in the following step. Similar to the standard auxiliary loss, this weighting is a proxy for the hard decision of whether to keep or drop that sample. That decision has the same non-differentiable indicator structure as DLB.

We propose to use a straight-through estimator to make the decision to keep or drop the sample directly during training of the scoring model. Rather than learning soft weights that are thresholded afterwards, the scoring model would produce a hard filtering decision in the forward pass using top-B, with the downstream gradient signal flowing back through a surrogate gradient on the indicator. This makes the training objective of the scoring model match what it is actually used for, selecting a subset of the data, instead of optimizing a soft proxy. This is analogous to how $\mathcal{L}_{\text{DLB}}$ makes the load-balancing objective match the hard counts rather than the soft routing probabilities.

In practice, we would score a candidate batch of size $B' > B$ and keep a filtered batch of exactly B examples in the forward pass. This will come with an additional cost. With a rectangular-window STE, gradients are not limited to the B selected examples, since unselected examples whose scores fall inside the rect window around the top-B threshold also receive a surrogate gradient. This means that the scoring model update will need to backpropagate through some boundary examples that are not used in the hard filtered batch, making each FLYT step more expensive than a purely hard top-B filter. However, this cost is paid while training the scoring model, not while training the final large model on the resulting filtered dataset. FLYT is already designed to make this stage cheap. It trains on only a small fraction of the candidate pool and reports substantially lower compute than the corresponding DataComp medium CLIP training run [46]. By contrast, strong prior filtering methods such as DFN and HYPE rely on training larger scale filtering networks or hyperbolic vision-language filtering models before scoring the pool [8, 20]. We therefore expect the

extra rect-window backward cost to be negligible relative to the cost of full-scale pretraining, and for the resulting method to remain substantially cheaper than these heavier filtering pipelines.

Making the filter aware of the sampling procedure

FLYT turns scores into a dataset with Soft Cap Sampling (SCS) [46], a procedure that samples examples according to their scores while applying a repetition penalty, so that high-scoring examples can be repeated but are not over-represented. SCS is currently a separate step applied after the scoring model is trained.

We propose to replace this separate sampling step with a discrete repetition objective. The final dataset is not really a binary filter/no-filter decision for each example. It is an integer allocation problem in which each example i receives a repetition count $r_i \in \{0, \dots, R_{\max}\}$ under a total sample budget $\sum_i r_i = K$. This objective can still be represented as a sum of indicators over possible repetition levels. For example, if a_i is the learned score of example i and $h(n)$ is an increasing penalty for using the n-th repetition, then one could write

$$r_i = \sum_{n=1}^{R_{\max}} \mathbb{1}_{\{a_i \geq \tau + h(n)\}},$$

so each additional repetition of the same example has to satisfy a stricter condition. The threshold τ controls the total budget, increasing it decreases $\sum_i r_i$, and it is set so that $\sum_i r_i = K$. This makes top-K filtering the special case $R_{\max} = 1$, and from this perspective SCS is a hand-designed approximation to a more general repetition-penalty rule. The complications are that the repetition penalty may need to depend on scale, score calibration, and diversity, and we cannot optimize the full large-scale allocation during every scoring model update. A practical direction may be to train or evaluate such discrete repetition policies at smaller scales, learn a scaling rule for the marginal value of additional repetitions, and then deploy that rule at the target scale by applying the learned repetition rule to the full score list. The rect STE would again apply to the indicators near the threshold, but the resulting objective is harder than keep-or-drop filtering because it must learn both which examples to include and how many times to repeat them.

Bibliography

[1] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv:1308.3432*, 2013.

[2] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, 2020.

[3] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 conference of the north American chapter of the association for computational linguistics: Human language technologies, volume 1 (long and short papers)*, 2019.

[4] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv:1803.05457*, 2018.

[5] Damai Dai, Chengqi Deng, Chenggang Zhao, RX Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, et al. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. In *Association for Computational Linguistics (ACL)*, 2024.

[6] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv:2412.19437*, 2024.

[7] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026.

[8] Alex Fang, Albin Madappally Jose, Amit Jain, Ludwig Schmidt, Alexander Toshev, and Vaishaal Shankar. Data filtering networks. *arXiv:2309.17425*, 2023.

[9] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 2022.

[10] Seokjin Go and Divya Mahajan. MOETUNER: Optimized mixture of expert serving with balanced expert placement and token routing. *arXiv:2502.06643*, 2025.

[11] Hongcan Guo, Haolang Lu, Guoshun Nan, Bolun Chu, Jialin Zhuang, Yuan Yang, Wenhao Che, Xinye Cao, Sicong Leng, Qimei Cui, et al. Advancing expert specialization for better MoE. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[12] Jiaao He, Jidong Zhai, Tiago Antunes, Haojie Wang, Fuwen Luo, Shangfeng Shi, and Qin Li. FasterMoE: Modeling and optimizing training of large-scale dynamic pre-trained models. In *ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 120–134, 2022.

[13] Alex Henry, Prudhvi Raj Dachapally, Shubham Shantaram Pawar, and Yuxuan Chen. Query-key normalization for transformers. In *Findings of the Association for Computational Linguistics: EMNLP*, 2020.

[14] Chenghao Hu, Yufei Kang, and Baochun Li. Communication-efficient MoE fine-tuning with locality-aware expert placement. In *IEEE International Conference on Distributed Computing Systems (ICDCS)*, pages 166–176, 2025.

[15] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Binarized neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[16] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, et al. Tutel: Adaptive mixture-of-experts at scale. *Proceedings of Machine Learning and Systems (MLSys)*, 2023.

[17] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv:2401.04088*, 2024.

[18] Keller Jordan, Jeremy Bernstein, Brendan Rappazzo, @fernbear.bsky.social, Vlado Boza, Jiacheng You, Franz Cesista, Braden Koszarsky, and @Grad62304977. modded-nanogpt: Speedrunning the NanoGPT baseline. `https://github.com/KellerJordan/modded-nanogpt`, 2024.

[19] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. `https://kellerjordan.github.io/posts/muon/`, 2024.

[20] Wonjae Kim, Sanghyuk Chun, Taekyung Kim, Dongyoon Han, and Sangdoo Yun. HYPE: Hyperbolic entailment filtering for underspecified images and texts. In *European Conference on Computer Vision (ECCV)*, pages 247–265, 2024.

[21] Jakub Krajewski, Jan Ludziejewski, Kamil Adamczewski, Maciej Pióro, Michał Krutul, Szymon Antoniak, Kamil Ciebiera, Krystian Król, Tomasz Odrzygóźdź, Piotr Sankowski, et al. Scaling laws for fine-grained mixture of experts. *arXiv:2402.07871*, 2024.

[22] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*, 2021.

[23] Hector J Levesque, Ernest Davis, and Leora Morgenstern. The winograd schema challenge. *KR*, 2012.

[24] Mike Lewis, Shruti Bhosale, Tim Dettmers, Naman Goyal, and Luke Zettlemoyer. BASE layers: Simplifying training of large, sparse models. In *International Conference on Machine Learning (ICML)*, 2021.

[25] Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, Sedrick Keh, Kushal Arora, Saurabh Garg, Rui Xin, Niklas Muennighoff, Reinhard Heckel, Jean Mercat, Mayee Chen, Suchin Gururangan, Mitchell Wortsman, Alon Albalak, Yonatan Bitton, Marianna Nezhurina, Amro Abbas, Cheng-Yu Hsieh, Dhruba Ghosh, Josh Gardner, Maciej Kilian, Hanlin Zhang, Rulin Shao, Sarah Pratt, Sunny Sanyal, Gabriel Ilharco, Giannis Daras, Kalyani Marathe, Aaron Gokaslan, Jieyu Zhang, Khyathi Chandu, Thao Nguyen, Igor Vasiljevic, Sham Kakade, Shuran Song, Sujay Sanghavi, Fartash Faghri,

Sewoong Oh, Luke Zettlemoyer, Kyle Lo, Alaaeldin El-Nouby, Hadi Pouransari, Alexander Toshev, Stephanie Wang, Dirk Groeneveld, Luca Soldaini, Pang Wei Koh, Jenia Jitsev, Thomas Kollar, Alexandros G. Dimakis, Yair Carmon, Achal Dave, Ludwig Schmidt, and Vaishaal Shankar. Datacomp-lm: In search of the next generation of training sets for language models. *arXiv:2406.11794*, 2024.

[26] Boan Liu, Liang Ding, Li Shen, Keqin Peng, Yu Cao, Dazhao Cheng, and Dacheng Tao. Diversifying the mixture-of-experts representation for language models with orthogonal optimizer. *arXiv:2310.09762*, 2023.

[27] Liyuan Liu, Jianfeng Gao, and Weizhu Chen. Sparse backpropagation for MoE training. *arXiv:2310.00811*, 2023.

[28] Liyuan Liu, Young Jin Kim, Shuohang Wang, Chen Liang, Yelong Shen, Hao Cheng, Xiaodong Liu, Masahiro Tanaka, Xiaoxia Wu, Wenxiang Hu, et al. GRIN: GRadient-INformed MoE. *arXiv:2409.12136*, 2024.

[29] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[30] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 2381–2391, 2018.

[31] Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Evan Pete Walsh, Oyvind Tafjord, Nathan Lambert, et al. OLMoE: Open mixture-of-experts language models. In *International Conference on Learning Representations (ICLR)*, 2025.

[32] Taishi Nakamura, Satoki Ishikawa, Masaki Kawamura, Takumi Okamoto, Daisuke Nohara, Jun Suzuki, and Rio Yokota. Optimal sparsity of mixture-of-experts language models for reasoning tasks. In *International Conference on Learning Representations (ICLR)*, 2026.

[33] Xiaonan Nie, Qibin Liu, Fangcheng Fu, Shenhan Zhu, Xupeng Miao, Xiaoyang Li, Yang Zhang, Shouda Liu, and Bin Cui. LSH-MoE: Communication-efficient MoE training via locality-sensitive hashing. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[34] Ashwinee Panda, Vatsal Baherwani, Zain Sarwar, Benjamin Thérien, Sambit Sahu, Tom Goldstein, and Supriyo Chakraborty. Dense backpropagation improves training for sparse mixture-of-experts. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[35] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc-Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: Long papers)*, 2016.

[36] Haotong Qin, Ruihao Gong, Xianglong Liu, Mingzhu Shen, Ziran Wei, Fengwei Yu, and Jingkuan Song. Forward and backward information retention for accurate binary neural networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[37] Zihan Qiu, Zeyu Huang, Bo Zheng, Kaiyue Wen, Zekun Wang, Rui Men, Ivan Titov, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Demons in the detail: On implementing load balancing loss for training specialized mixture-of-expert models. In *Association for Computational Linguistics (ACL)*, 2025.

[38] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 2020.

[39] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. In *International Conference on Machine Learning (ICML)*, 2022.

[40] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 conference on empirical methods in natural language processing*, 2016.

[41] Siva Reddy, Danqi Chen, and Christopher D Manning. CoQA: A conversational question answering challenge. *Transactions of the Association for Computational Linguistics*, 2019.

[42] Melissa Roemmele, Cosmin Adrian Bejan, and Andrew S Gordon. Choice of plausible alternatives: An evaluation of commonsense causal reasoning. In *AAAI spring symposium: logical formalizations of commonsense reasoning*, pages 90–95, 2011.

[43] Stephen Roller, Sainbayar Sukhbaatar, Jason Weston, et al. Hash layers for large sparse models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[44] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 2021.

[45] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.

[46] Mikey Shechter and Yair Carmon. Filter like you test: Data-driven data filtering for CLIP pretraining. *arXiv:2503.08805*, 2025.

[47] David So, Wojciech Mańke, Hanxiao Liu, Zihang Dai, Noam Shazeer, and Quoc V Le. Searching for efficient transformers for language modeling. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[48] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on machine learning research*, 2023.

[49] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 2024.

[50] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. CommonsenseQA: A question answering challenge targeting commonsense knowledge. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 2019.

[51] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[52] Lean Wang, Huazuo Gao, Chenggang Zhao, Xu Sun, and Damai Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. *arXiv:2408.15664*, 2024.

[53] Tianwen Wei, Bo Zhu, Liang Zhao, Cheng Cheng, Biye Li, Weiwei Lü, Peng Cheng, Jianhao Zhang, Xiaoyu Zhang, Liang Zeng, et al. Skywork-MoE: A deep dive into training techniques for mixture-of-experts language models. *arXiv:2406.06563*, 2024.

[54] Zijie Yan, Hongxiao Bai, Xin Yao, Dennis Liu, Tong Liu, Hongbin Liu, Pingtian Li, Evan Wu, Shiqing Fan, Li Tao, et al. Scalable training of mixture-of-experts models with Megatron Core. *arXiv:2603.07685*, 2026.

[55] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv:2505.09388*, 2025.

[56] Jinghan Yao, Quentin Anthony, Aamir Shafi, Hari Subramoni, and Dhabaleswar K. Panda. Exploiting inter-layer expert affinity for accelerating mixture-of-experts model inference. *arXiv:2401.08383*, 2024.

[57] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th annual meeting of the association for computational linguistics*, 2019.

[58] Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin Chen, Chengxing Xie, Cunxiang Wang, Da Yin, Hao Zeng, Jiajie Zhang, et al. GLM-4.5: Agentic, reasoning, and coding (ARC) foundation models. *arXiv:2508.06471*, 2025.

[59] Mingshu Zhai, Jiaao He, Zixuan Ma, Zan Zong, Runqing Zhang, and Jidong Zhai. SmartMoE: Efficiently training sparsely-activated models through combining offline and online parallelization. In *USENIX Annual Technical Conference (USENIX ATC)*, pages 961–975, 2023.

[60] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[61] Mohan Zhang, Pingzhi Li, Jie Peng, Mufan Qiu, and Tianlong Chen. Advancing MoE efficiency: A collaboration-constrained routing (C2R) strategy for better expert parallelism design. In *Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL)*, pages 6815–6825, 2025.

[62] Wanjun Zhong, Ruixiang Cui, Yiduo Guo, Yaobo Liang, Shuai Lu, Yanlin Wang, Amin Saied, Weizhu Chen, and Nan Duan. AGIEval: A human-centric benchmark for evaluating foundation models. In *Findings of the association for computational linguistics: NAACL 2024*, 2024.

[63] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M Dai, Quoc V Le, James Laudon, et al. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[64] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. ST-MoE: Designing stable and transferable sparse expert models. *arXiv:2202.08906*, 2022.

Appendix A

Direct Load Balancing: Experimental Details

A.1 Coefficient sweeps

Table A.1: Local coefficient sweep, $K = 16$ (part 1 of 2): $\mathcal{L}_{\text{DLB}}$, $\mathcal{L}_{\text{uncentered-DLB}}$, and $\mathcal{L}_{\text{MaxVio}}$. Each cell reports validation loss, MaxVio, and TotalVio.

LB coefficient α	$\mathcal{L}_{\text{DLB}}$			$\mathcal{L}_{\text{uncentered-DLB}}$			$\mathcal{L}_{\text{MaxVio}}$		
	Validation loss	MaxVio	TotalVio	Validation loss	MaxVio	TotalVio	Validation loss	MaxVio	TotalVio
10^{-4}	3.028	2.746	33.448	3.035	2.763	47.591	3.029	0.627	30.126
10^{-3}	3.028	0.667	5.807	3.046	2.633	58.938	3.295	0.175	5.663
10^{-2}	3.031	0.043	0.654	3.064	0.463	10.969	3.166	0.067	2.989
10^{-1}	3.044	0.011	0.223	3.277	0.552	7.079	3.240	0.009	0.195

Table A.2: Local coefficient sweep, $K = 16$ (part 2 of 2): $\mathcal{L}_{\text{MaxVio}^2}$, $\mathcal{L}_{\text{TotalVio}}$, and the auxiliary loss $\mathcal{L}_{\text{aux}}$.

LB coefficient α	$\mathcal{L}_{\text{MaxVio}^2}$			$\mathcal{L}_{\text{TotalVio}}$			$\mathcal{L}_{\text{aux}}$		
	Validation loss	MaxVio	TotalVio	Validation loss	MaxVio	TotalVio	Validation loss	MaxVio	TotalVio
10^{-4}	3.029	2.850	53.054	3.032	0.094	0.693		–	
10^{-3}	3.030	1.226	46.441	3.159	0.046	1.056	3.034	0.955	16.329
10^{-2}	3.029	0.376	19.856	3.263	0.066	0.987	3.031	0.086	2.022
10^{-1}	3.033	0.119	6.353	3.555	0.001	0.026	3.304	0.168	2.772

Table A.3: Local coefficient sweep, $K = 2$ (part 1 of 2): $\mathcal{L}_{\text{DLB}}$, $\mathcal{L}_{\text{uncentered-DLB}}$, and $\mathcal{L}_{\text{MaxVio}}$.

LB coefficient α	$\mathcal{L}_{\text{DLB}}$			$\mathcal{L}_{\text{uncentered-DLB}}$			$\mathcal{L}_{\text{MaxVio}}$		
	Validation loss	MaxVio	TotalVio	Validation loss	MaxVio	TotalVio	Validation loss	MaxVio	TotalVio
10^{-4}	3.138	4.727	70.632	3.141	4.376	72.616	3.229	2.499	77.028
10^{-3}	3.113	0.565	9.584	3.120	0.621	14.385	3.234	0.355	18.174
10^{-2}	3.128	0.169	0.978	3.155	0.519	7.719	3.175	0.108	2.038
10^{-1}	3.164	0.021	0.298	3.275	0.617	10.841	3.726	0.070	1.429

The following tables report the preliminary bandwidth sweeps for $K = 16$. These runs were produced in a different environment from the runs used in Table 1, so they are useful for studying

Table A.4: Local coefficient sweep, $K = 2$ (part 2 of 2): $\mathcal{L}_{\text{MaxVio}^2}$, the auxiliary loss $\mathcal{L}_{\text{aux}}$, and $\mathcal{L}_{\text{TotalVio}}$.

LB coefficient α	$\mathcal{L}_{\text{MaxVio}^2}$			$\mathcal{L}_{\text{TotalVio}}$			$\mathcal{L}_{\text{aux}}$		
	Validation loss	MaxVio	TotalVio	Validation loss	MaxVio	TotalVio	Validation loss	MaxVio	TotalVio
10^{-4}	3.170	7.122	97.893	3.122	0.337	1.865		–	
10^{-3}	3.148	2.456	77.739	3.403	0.093	1.775	3.249	1.136	15.854
10^{-2}	3.130	0.737	41.462	3.577	0.066	1.386	3.239	0.241	4.125
10^{-1}	3.143	0.212	10.148	4.927	0.154	1.766	3.192	0.028	0.538

sensitivity to h but are not included in the main comparison.

Table A.5: Local bandwidth sweep for $\mathcal{L}_{\text{DLB}}$ with LB coefficient $\alpha = 10^{-2}$, $K = 16$.

Bandwidth h	Validation Loss	MaxVio	TotalVio
0.1	3.032	0.058	0.756
0.5	**3.030**	**0.045**	**0.605**
1.0	3.032	0.048	0.769
2.0	3.031	0.067	1.157
5.0	3.036	0.076	1.305
10.0	3.036	0.105	1.874

Table A.6: Local bandwidth sweep for $\mathcal{L}_{\text{MaxVio}}$ with LB coefficient $\alpha = 10^{-3}$, $K = 16$.

Bandwidth h	Validation Loss	MaxVio	TotalVio
0.1	3.032	0.098	4.858
0.5	**3.029**	**0.093**	**4.254**
1.0	3.032	0.097	4.423
2.0	3.032	0.100	4.548
5.0	3.033	0.116	5.604
10.0	3.035	0.200	10.599

The sweeps show the same picture as the main text. At $K = 16$ the centered $\mathcal{L}_{\text{DLB}}$ reaches a MaxVio of 0.043 at $\alpha = 10^{-2}$ while keeping the validation loss within 0.001 of the best run, whereas the uncentered $\mathcal{L}_{\text{uncentered-DLB}}$ never balances well at any coefficient — at $\alpha = 10^{-2}$ its MaxVio is 0.463, an order of magnitude worse. This is the empirical signature of the analysis in Section 3.3: under the STE surrogate the centered and uncentered objectives are not equivalent, and only the centered one balances. The same pattern holds at $K = 2$.

A.2 Large-scale sweep

Table A.7: Local bandwidth sweep for $\mathcal{L}_{\text{TotalVio}}$, $K = 16$.

LB coefficient α	Bandwidth h	Validation Loss	MaxVio	TotalVio
10^{-5}	1.0	**3.028**	1.606	6.735
10^{-5}	5.0	3.032	2.216	8.695
10^{-5}	10.0	3.031	2.819	14.821
10^{-4}	1.0	3.031	0.099	0.710
10^{-4}	5.0	3.037	0.057	0.896
10^{-4}	10.0	3.037	0.108	1.518
10^{-3}	1.0	3.050	0.008	0.123
10^{-3}	5.0	3.049	0.015	0.296
10^{-3}	10.0	3.042	0.023	0.388
10^{-2}	1.0	3.154	0.006	0.063
10^{-2}	5.0	3.211	0.218	1.246
10^{-2}	10.0	3.157	0.200	0.865
10^{-1}	1.0	3.555	0.001	0.026

Table A.8: **Full large-scale sweep.** Downstream CORE metrics, validation loss, and balance metrics for all $E = 64$, $K = 8$ runs trained for 100B tokens. For $\mathcal{L}_{\text{aux}}$ and $\mathcal{L}_{\text{DLB}}$, the LB strength is the auxiliary-loss coefficient α, and for DeepSeek loss-free it is the bias update rate.

LB method	LB strength	STE width	Mean CORE loss	CORE accuracy	Validation loss	MaxVio	TotalVio
$\mathcal{L}_{\text{aux}}$	0.001	–	5.416	0.274	2.857	3.228	39.109
$\mathcal{L}_{\text{aux}}$	0.01	–	5.451	0.274	2.849	0.857	9.312
DeepSeek loss-free (softmax)	0.001	–	5.282	0.286	2.765	0.919	2.150
DeepSeek loss-free (softmax)	0.01	–	5.263	0.290	2.755	0.041	0.678
DeepSeek loss-free (sigmoid)	0.001	–	5.228	0.283	2.779	0.690	1.640
DeepSeek loss-free (sigmoid)	0.01	–	5.347	0.286	2.772	0.020	0.268
$\mathcal{L}_{\text{DLB}}$	0.001	0.1	5.351	0.287	2.762	2.346	35.339
$\mathcal{L}_{\text{DLB}}$	0.001	0.5	5.356	0.281	2.801	1.059	9.677
$\mathcal{L}_{\text{DLB}}$	0.001	1	5.423	0.284	2.808	0.683	5.556
$\mathcal{L}_{\text{DLB}}$	0.001	2	5.408	0.280	2.850	0.582	6.024
$\mathcal{L}_{\text{DLB}}$	0.001	5	5.408	0.282	2.830	0.195	1.823
$\mathcal{L}_{\text{DLB}}$	0.01	0.1	5.124	0.279	2.753	0.331	3.359
$\mathcal{L}_{\text{DLB}}$	0.01	0.5	5.245	0.285	2.752	0.087	1.188
$\mathcal{L}_{\text{DLB}}$	0.01	1	5.357	0.284	2.756	0.053	0.931
$\mathcal{L}_{\text{DLB}}$	0.01	2	5.360	0.282	2.778	0.089	1.401
$\mathcal{L}_{\text{DLB}}$	0.01	5	5.406	0.280	2.838	0.138	1.471